

分类号：TP311

单位代码：10110

学 号：S2007001

中 北 大 学
硕 士 学 位 论 文
(学 硕)

基于深度学习的源代码多类型漏洞
检测框架

基于深度学习的源代码多类型漏洞检测框架 王守梁 中北大学

硕士研究生 王守梁

指导教师 宋礼鹏

学科专业 计算机科学与技术

2023 年 12 月 01 日

图书分类号 TP311 密级 非密

UDC ^{注 1} 004.7

硕 士 学 位 论 文

基于深度学习的

源代码多类型漏洞检测框架

王守梁

(作者姓名)

指导教师(姓名、职称) 宋礼鹏 教授

申请学位级别 工学硕士

专业名称 计算机科学与技术

论文提交日期 2023 年 3 月 27 日

论文答辩日期 2023 年 11 月 24 日

学位授予日期 年 月 日

论文评阅 郭春 王赧

答辩委员会主席 石洪波

2023 年 12 月 01 日

注 1: 注明《国际十进分类法 UDC》的分类

原创性声明

本人郑重声明：所呈交的学位论文，是本人在指导教师的指导下，独立进行研究所取得的成果。除文中已经注明引用的内容外，本论文不包含其他个人或集体已经发表或撰写过的科研成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律责任由本人承担。

论文作者签名： 王守梁 日期： 2023年12月01日

关于学位论文使用权的说明

本人完全了解中北大学有关保管、使用学位论文的规定，其中包括：
①学校有权保管、并向有关部门送交学位论文的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存学位论文；③学校可允许学位论文被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换学位论文；⑤学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

签 名： 王守梁 日期： 2023年12月01日
导师签名： 宋一鹏 日期： 2023年12月01日

基于深度学习的源代码多类型漏洞检测框架

摘要

随着大数据时代的来临，软件代码量级攀升，软件更加复杂化与多样化，漏洞的危害性被不断放大，任何一个微小的漏洞被攻击者利用都可能会导致整个软件产品的崩溃。此外，漏洞还具有不可避免性。综上所述，软件源代码漏洞检测逐渐成为信息安全领域的研究热点，如何进行高效率与高精度的漏洞检测已然成为安全领域的一个重要挑战。

然而，目前工业生产环节主流的仍是传统的漏洞检测技术，对专业人士及专业知识的过高追求，增加了漏洞检测成本的同时，也很大程度上限制了检测效率，开始不适应大数据时代软件和漏洞复杂性的增加。基于深度学习的漏洞检测开始逐步引起学术界和业界的广泛关注，经过近几年的探索，深度学习技术在漏洞检测领域的能力得到了一定程度的验证。本文针对程序源代码的两类不同的中间表示形式，分别提出了对应的函数级源代码漏洞检测方案。本文研究的主要工作内容以及创新点如下：

（1）提出了基于序列的函数级源代码漏洞检测方案：融合多种词向量技术，首次引入分层注意力网络与多头文本卷积神经网络，设计了包含二者的漏洞检测模型。此外，通过变换多层注意力机制的基本网络单元结构，进一步提升模型性能。

（2）提出了基于图与序列的函数级源代码漏洞检测方案：综合学习程序语义和结构信息来识别漏洞，以自动化的方式捕获了比现有方法更综合的特征。其中，基于扩展后得到的更综合的代码图形式，提出了全新的图嵌入策略，在多环节引入基于编程语言的预训练语言模型 CodeBert 来代替传统的词向量模型进行信息编码。

（3）基于现有漏洞数据集开展漏洞检测实验，分别将以上两种漏洞检测方案与已有研究成果进行对比，实验结果表明本文提出的两种漏洞检测方案较已有成果具有性能上的优越性。

关键词：漏洞检测，多类型，注意力机制，源代码，词向量，深度学习

Multitype vulnerability detection architecture for source code based on deep learning

Abstract

With the advent of the era of big data, the magnitude of software code has climbed, software has become more complex and diverse, The harm of vulnerabilities is constantly amplified, and any small vulnerability exploited by an attacker may lead to the collapse of the entire software product. In addition, vulnerabilities are inevitable. In summary, software source code vulnerability detection has gradually become a research hotspot in the field of information security, how to conduct efficient and high-precision vulnerability detection has become an important challenge in the security field.

However, at present, the mainstream of industrial production is still traditional vulnerability detection technology. The excessive pursuit of professionals and professional knowledge has increased the cost of vulnerability detection, but also greatly limited the detection efficiency. It is beginning to adapt to the increase in software and vulnerability complexity in the era of big data. Vulnerability detection based on deep learning has gradually attracted widespread attention in academia and the industry. Through exploration in recent years, the ability of deep learning technology in the field of vulnerability detection has been verified to a certain extent. This article proposes corresponding function level source code vulnerability detection schemes for two different intermediate representations of program source code. The main work content and innovative points studied in this article are as follows:

(1) A function level source code vulnerability detection scheme based on sequence has been proposed, which integrates multiple word vector techniques. For the first time, a hierarchical attention network and a multi head text convolutional neural network were introduced, and a vulnerability detection model containing both was designed. In addition, by transforming the basic network unit structure of the multi-layer attention mechanism, the performance of the model is further improved.

(2) A function level source code vulnerability detection scheme based on graph and sequence has been proposed, which comprehensively learns program semantics and structural information to identify vulnerabilities and automatically captures more comprehensive features than existing methods. Among them, based on the more comprehensive code graph form obtained after expansion, a new graph embedding strategy is proposed, and a pre trained language model CodeBert based on programming language is introduced in multiple stages to replace traditional word vector models for information encoding.

(3) Based on existing vulnerability datasets, vulnerability detection experiments were conducted, and the above two vulnerability detection schemes were compared with existing research results. The experimental results showed that the two vulnerability detection schemes proposed in this paper have superior performance compared to existing achievements.

Keywords: vulnerability detection, multi-type, attention mechanism, source code, word vector, deep learning

目 录

第一章 绪论.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.2.1 传统漏洞检测研究现状.....	2
1.2.2 漏洞数据集研究现状.....	3
1.2.3 基于传统机器学习的漏洞检测研究现状.....	4
1.2.4 基于深度学习的漏洞检测研究现状.....	5
1.3 论文的研究内容.....	6
1.4 论文的组织结构.....	7
第二章 基于序列的函数级源代码漏洞检测方案的设计.....	9
2.1 输入层.....	10
2.1.1 Word2vec.....	11
2.1.2 FastText.....	13
2.1.3 GloVe.....	13
2.1.4 ELMo.....	15
2.2 多层注意力网络（HAN）.....	16
2.2.1 网络结构.....	17
2.2.2 基本原理.....	17
2.3 文本卷积神经网络（Text-CNN）.....	19
2.3.1 网络结构.....	19

2.3.2 基本原理	20
2.4 漏洞检测层	21
2.5 实现流程	21
2.5.1 词向量生成	21
2.5.2 训练与检测	22
2.6 本章小结	23
第三章 基于图与序列的函数级源代码漏洞检测方案的设计	24
3.1 基于图的特征学习	24
3.1.1 图的生成	24
3.1.2 节点嵌入	26
3.1.3 边嵌入	27
3.1.4 图神经网络	28
3.1.5 图嵌入	29
3.2 基于序列的特征学习	30
3.3 特征融合	30
3.4 分类器	31
3.5 本章小结	31
第四章 实验设计与结果分析	32
4.1 实验环境	32
4.2 数据集	32
4.2.1 基于序列的函数级源代码漏洞检测方案数据集	32

4.2.2 基于图与序列的函数级源代码漏洞检测方案数据集.....	33
4.3 评估指标	34
4.3.1 基于序列的函数级源代码漏洞检测方案评估指标.....	34
4.3.2 基于图与序列的函数级源代码漏洞检测方案评估指标.....	36
4.4 实验与结果分析.....	36
4.4.1 基于序列的函数级源代码漏洞检测方案实验设计	36
4.4.2 基于序列的函数级源代码漏洞检测方案结果与分析.....	38
4.4.3 基于图与序列的函数级源代码漏洞检测方案实验设计.....	41
4.4.4 基于图与序列的函数级源代码漏洞检测方案结果与分析	42
4.5 本章小结	47
第五章 总结与展望.....	48
5.1 全文工作总结	48
5.2 未来工作展望.....	48
参考文献	
攻读硕士学位期间所取得的成就	
致谢	

第一章 绪论

1.1 研究背景与意义

进入 5G 时代以来, 互联网技术及其应用飞速发展, 云计算、大数据以及人工智能技术开始渗透到医疗、教育、出行等人类生活的方方面面, 数字经济蓬勃发展, 网络基础设施全面覆盖, 智能化服务的类型逐渐多元、形式逐渐多样, 极大程度的方便了人类的日常生活。此外, 智慧城市、智慧政务以及智慧管理等新概念的提出及落地, 创新了城市运行管理方式、简化了政府办公程序、优化了企业架构, 极大程度的改善了人类的生产方式。互联网逐渐迈入智慧物联阶段, 据统计, 截至 2022 年 6 月底, 全球互联网用户数达到 53.86 亿人, 其中我国互联网用户数占比接近五分之一, 接近 11 亿人; 我国互联网普及率 74.4%, 远超全球平均 65%。

各项数字技术的发展在给人们带来方便的同时, 也带来了信息安全隐患。小到外卖餐饮的在线点单, 大到智慧城市管理中资源的决策调度, 越来越多的行业开始使用软件作为其业务载体, 代码已经成为支撑社会正常运转的最基本的元素之一, 软件安全已经直接影响到人们的日常生活和社会的正常运转, 信息技术快速发展所带来的漏洞数量的激增逐渐成为当今社会的基础性和根本性问题。软件源代码漏洞检测一直是软件安全领域的一个重要挑战^{[1][2][3]}。2022 年, 缺陷名录网站(NVD)全年的漏洞总数为 6515 个, 预期年增长率至少提升了 170%; 公共漏洞披露平台(CVE)一整年的漏洞总个数为 10703 个, 预期的年增长率高达 70%。政府单位、银行以及科技巨头企业已成为攻击者漏洞利用的重点^[4], 2022 年, 黑客利用 SS7 漏洞窃取验证码造成西班牙电信公司客户资金流失、苹果设备出现任意代码执行缓冲区溢出漏洞、博通 WiFi 芯片出现堆溢出漏洞....., 不法分子每年利用软件漏洞引发的安全事故在全球造成的直接经济损失高达上千亿美元^[5]。此外, 当今国际形势严峻, 漏洞安全问题在很大程度上影响着国家安全问题, 已成为大国博弈的关键领域。

上述数据及漏洞安全事件说明, 漏洞安全问题日益严重, 然而, 传统的漏洞检测技术存在由其工作模式带来的难以避免的问题: 一方面, 无法适应大数据时代代码与漏

洞数量级的指数级增加，无法在实际软件开发环节中及时且准确的修复高复杂性与多样性的新型漏洞；另一方面需要专业人员花费较多的人力，其在软件的开发过程中一直占用了较高的成本，研究表明，软件漏洞识别及修复所耗费的人力物力财力极高，约占整个软件开发总成本的 50%-70%^[6]。高效的自动化漏洞检测技术的需求攀升。近年来，开源软件飞速发展，开源项目托管网站(GitHub)与各大漏洞库的实时更新为研究人员提供了越来越多的代码量与漏洞信息，这些代码数据为基于学习的自动化漏洞检测研究提供了充分的数据基础。此外，计算能力的提升使得深度学习技术得到了快速发展，其在影像识别、自然语言处理、音视频识别与鉴伪等领域均获得了突破性进展。

基于此，研究者开始将深度学习技术引入代码领域来替代传统方法，期待解决传统方法无法解决的诸多问题。经过近几年的探索，深度学习技术在漏洞检测领域的能力得到了一定程度的验证，本研究考虑借助现有的漏洞数据集，提出了两种新型的漏洞检测方案，来实现高效的自动化漏洞挖掘。

1.2 国内外研究现状

1.2.1 传统漏洞检测研究现状

源代码的漏洞检测是软件系统设计中极为重要的一环，是工业软件开发过程必不可少的一道工序。其通过特定的分析工具分析软件代码的执行过程，查找并且定位设计缺陷或者能够造成运行故障的代码点。传统的漏洞检测技术根据代码是否运行分为静态分析技术^[7]与动态分析技术^[8]两种。其中，静态分析技术常使用在编码开发阶段，通过扫描源代码文本，利用代码词法、语法、数据流及控制流等信息生成源代码结构信息，辅以漏洞检测，整个过程无需软件运行；动态分析技术常使用在软件的测试阶段，通过关注执行路径、程序状态等信息来检测漏洞，整个过程需运行软件。目前，有许多投入使用且效果良好的漏洞检测工具：以 Coverity^[9]与 Klocwork^[10]等为代表的静态分析工具；以 libFuzzer^[11]及 AFL 等为代表的基于模糊测试的动态分析工具；以 KLEE^[12]、S2E^[13]等为代表的基于符号执行的动态分析工具；以 angr^[14]为代表的结合静态分析与动态符号执行的混合分析工具。

此外，近些年也陆续研发出一些基于传统理论的自动化或半自动化的漏洞检测工具：Digtool^[15]自动捕捉 Windows 系统程序执行漏洞，Rapidscan^[16]自动调用 nmap、dnsrecon 和 wafw00f 等多个扫描工具融合扫描结果。

由于传统漏洞检测技术无法突破其固有的工作模式定式，注定无法适应大数据时代下漏洞类型及数量的激增。静态分析过于依赖专家知识等主观性的因素，始终无法获得令人满意的准确率；动态分析则存在符号执行路径爆炸、约束求解难以及模糊测试过于依赖种子质量、偶然性大等诸多问题。传统的漏洞检测技术研究已逐渐淡出漏洞安全领域的研究中心。

1.2.2 漏洞数据集研究现状

近年来，该领域研究方向有所转变，由被动检测转向漏洞代码的主动预测。基于学习的漏洞检测方案逐渐被提出。漏洞数据集的构建与选取开始成为漏洞检测领域的核心研究工作之一，漏洞数据集很大程度上决定基于学习漏洞检测方案的训练效果，进而影响模型的检测性能。当前，在漏洞检测领域缺乏标准统一的漏洞数据集，各类研究常使用自构建的数据集来评测漏洞检测模型的性能。基于此，本文梳理了漏洞数据集构建的考虑要素，并将其分为以下三个方面：1.漏洞代码来源 2.样本粒度选择 3.标签处理方式。具体细分如图 1-1 所示。

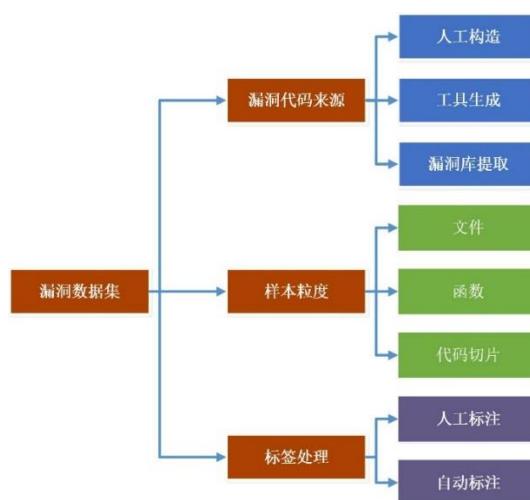


图 1-1 漏洞数据集构造方法分类

Fig1-1 Classification of vulnerability dataset construction methods

基于此，对已有的漏洞数据集进行了分类梳理，主要研究成果如表 1-1 所示。

表 1-1 漏洞数据集研究成果

Tab1-1 Research achievements in vulnerability datasets

作者	漏洞代码来源	样本粒度	标签处理	语言	漏洞类型数
Duan X, Wu J, Ji S ^[17]	人工构造	函数	人工标注	C/C++	2
Ghaffarian SM, Shahriari HR ^[18]	人工构造	文件	人工标注	Java	7
Zou D, Wang S, Xu S ^[19]	漏洞库提取	代码切片	自动标注	C/C++	40
Li J, He P, Zhu J ^[20]	漏洞库提取	代码切片	自动标注	Java	——
Russell R, Kim L, Hamilton L ^[21]	工具生成	函数	人工标注	C	149

然而，自构建数据集存在多方面问题：一是漏洞代码来源及粒度选择存在差异，这就导致各个漏洞检测模型性能的横向比较十分困难；二是自构建过程存在不合理操作，影响检测模型的训练表现。

1.2.3 基于传统机器学习的漏洞检测研究现状

传统的漏洞检测通过人为分析代码数据来获得漏洞相关信息，对从业人员的代码数据分析能力要求极高，而基于学习的方法可以很好的从海量的数据中发现学习规律；基于代码文本的统计学研究发现，代码语言与自然语言在数学上具有相似的统计学特性。鉴于以上两个方面，面对复杂代码的漏洞检测问题，研究者开始逐步引入传统的机器学习方法进行漏洞检测。

VCCFinder^[22]选择代码修改与开发者行为作为度量元，使用 one-hot 对样本特征值进行编码后使用支持向量机进行分类；Walden^[23]等人利用 12 个代码复杂度度量元构建漏洞检测模型后使用随机森林进行分类；Yamaguchi^[24]等人提出的 Chucky 系统结合了机器学习与静态代码分析用来检测源代码中的检查丢失，后又提出代码属性图概念辅

以检测漏洞；pang^[25]等人使用 n-gram 分析与统计特征选择技术来进行代码漏洞特征的提取，并辅以支持向量机进行分类检测。

然而，上述传统的机器学习方法过分依赖专家对特征进行人工构造作为输入，受限于特征构造质量的好坏，尚未完全摆脱专业人员的介入，距离自动化漏洞检测要求较远。此外，传统机器学习方法深层特征的挖掘能力有限，实验结果普遍存在过高的误报率与漏报率。

1.2.4 基于深度学习的漏洞检测研究现状

近年来，鉴于深度学习在自然语言处理、图像识别等领域取得的巨大成果，深度学习与漏洞检测的结合成为研究热点。研究者发现深度学习方法在漏洞深层特征的挖掘上具有很大优势，现有的研究成果已经证明，深度学习相比传统机器学习在漏洞检测领域具有无可比拟的优势。使用深度学习方法进行漏洞检测离不开漏洞检测模型的选取与构建，其是该研究领域的核心部分。

在源代码漏洞检测领域，现有的综述已从不同的角度对漏洞检测领域的研究进行了全面的总结^{[26][27][28][29][30]}。本节将聚焦本文研究工作的三个关键方面：源代码的中间表示形式、词向量技术以及神经网络模型。并从以上三个角度对已有的漏洞检测工作分别进行总结回顾。

首先，由于代码具有各个维度的特征，如何对代码进行表征成为漏洞检测模型结构设计的首要考量因素，目前研究主要分为基于代码序列的漏洞检测与基于代码图结构的漏洞检测两类。前者将源代码看作一类特殊的自然语言，使用自然语言处理领域的成熟模型解决漏洞检测任务，聚焦代码的语义信息，Wang^[31]、Russell^[32]及 Li^[33]等人的研究在源代码序列基础上借助不同的神经网络模型实现漏洞检测。后者抽取代码对应的图表示并借助图神经网络解决漏洞检测任务，聚焦代码的结构信息，Chakraborty^[34]、Li^[35]及 Dong^[36]等人的研究将源代码转换成 CPG 图实现漏洞检测。

其次，词向量技术用于生成神经网络可处理的实值向量。其选取决定着漏洞代码的上下文信息的提取，进而影响模型的检测性能。目前，漏洞检测研究普遍采用 Word2Vec 生成代码向量，用于训练深度学习模型。

最后，神经网络模型主要分为两类，分别对应两种中间表示形式。包括卷积神经网络(CNN)、多层感知器(MLP)、深度置信网络(DBN)、长短期记忆网络(LSTM)、及门控单元网络(GRU)等在内的多种神经网络模型被应用在基于序列的漏洞检测并取得了良好的检测效果；包括门控图神经网络(GGNN)、图卷积神经网络(GCN)等在内的多种图神经网络模型被应用在基于图的漏洞检测，用于生成图向量。成果汇总见表 1-2。

表 1-2 漏洞检测神经网络模型汇总

Tab1-2 Summary of Vulnerability Detection Neural Network Models	
神经网络模型	研究成果
深度置信网络	Wang et al. ^[31]
卷积神经网络	Russell et al. ^[32] , Harer et al. ^[36] , Lee et al. ^[36] , and Wu et al. ^[39]
多层感知器	Lin et al. ^[40] and Shar and Tan ^[41]
长短期记忆网络	Li et al. ^[33] , Lin et al. ^[42] , and Lin et al. ^[43]
门控单元网络	Lin et al. ^[44]
门控图神经网络	Chakraborty et al. ^[34] , Li et al. ^[45] and Wang et al. ^[46]
图卷积神经网络	Dong et al. ^[47]

综上所述，该领域的研究尚处在起步阶段，研究数量呈现逐年增加的趋势，尚存在较大的研究空间供学者探究。

1.3 论文的研究内容

本文将源代码漏洞检测工作视为分类任务，旨在识别源代码中对应函数是否存在漏洞。数据样本的形式化定义如下： $\{(x_i, y_i) | x_i \in X, y_i \in Y\}_{i=1}^N$ 。其中， X 为源程序数据集， $Y=\{0,1,\dots\}$ ，代表样本标签，标签大于 0 代表数据样本含有对应类型的漏洞，标签 0 代表数据样本不含漏洞， N 表示样本的总个数。本研究的最终目标为学习一个映射函数 $f: X \rightarrow Y$ 来确定给定的源代码是否为漏洞代码。本文针对漏洞的两类中间表示形式，分别提出了对应的函数级源代码漏洞检测方案。借助现有研究的数据集分别开展实验验证了所提方案的有效性。相关研究内容如下：

(1) 提出了基于序列的函数级源代码漏洞检测方案，融合包括 word2vec^[50]、glove^[51]、fasttext^[52]及预训练模型 ELMo^[53]多种词向量技术，首次引入分层注意力网络

（Hierarchical Attention Network^[48]，HAN）与多头文本卷积神经网络（Mutil-Head Text-CNN^[49]），设计了包含二者的神经网络模型。

（2）提出了基于图与序列的函数级源代码漏洞检测方案，综合学习程序语义和结构信息来识别漏洞，以自动化的方式捕获了比现有方法更综合的特征。其中，在多环节引入基于编程语言的预训练语言模型 CodeBert 来代替传统的词向量模型进行信息编码。

（3）基于现有漏洞数据集开展漏洞检测实验，分别将以上两种漏洞检测方案与已有研究成果进行对比，实验结果表明本文提出的两种漏洞检测方案较已有成果具有性能上的优越性。

1.4 论文的组织结构

本论文对使用深度学习进行源代码漏洞检测问题进行研究，首先介绍了漏洞检测领域相关背景及意义，进一步介绍了漏洞检测领域的研究趋势，提出了全新的漏洞检测模型。全文共六章，各章主要内容概述如下：

第一章：绪论。

介绍了本文的研究背景，具体分析了传统漏洞检测、基于传统机器学习的漏洞检测与基于深度学习的漏洞检测的研究现状，并对本文的研究内容与组织结构进行了概述

第二章：基于序列的函数级源代码漏洞检测方案的设计与实现。

详细阐述了方案的结构与原理。首先，介绍了输入层所使用的词向量模型，详细阐明了每个词向量模型的应用原理。然后，阐述引入多层注意力网络与文本卷积网络的相关依据，并分别详细介绍了两种模型的结构与原理。最后，介绍了漏洞检测层分类器的选择。此外，详细阐述了方案的实现流程。

第三章：基于图与序列的函数级源代码漏洞检测方案的设计与实现。

详细阐述了方案的结构与原理。首先，详细介绍了基于图的特征学习模块的实现原理，包括图的生成、节点嵌入、边嵌入以及图嵌入。然后，详细介绍了基于序列的特征学习模块的实现原理。最后，介绍了特征融合模块的实现原理与分类器的选择。输入层所使用的词向量模型，详细阐明了每个词向量模型的应用原理。此外，详细阐述

了方案的实现流程。

第四章：实验与分析。

详细阐述了两种方案的实验设计。分别介绍了两种方案对应的漏洞数据集与评估指标。后通过“提出问题—解决问题”的形式设计实验，验证了两种方案的检测性能。此外，本文通过控制单一变量的对照实验进一步验证了两方案各环节中改进方法的有效性。

第五章：总结与展望。

总结了本文主要的研究内容以及取得的相关成果，分析了当前研究的优势及不足，阐明了未来的研究方向与研究目标。

第二章 基于序列的函数级源代码漏洞检测方案的设计

本方案旨在基于合成数据集完成多类型的漏洞检测任务，代码中不同的关键字与语句类型对漏洞的影响有所不同。因此，在传统的循环神经网络基础上引入注意力机制同时关注影响漏洞的关键词与关键语句对提升漏洞检测模型的检测性能至关重要。HAN 神经网络模型是一种针对文本分类任务的层次注意力网络。其基于文档序列的不同部分对于文档分类的重要性不同，单词和句子的重要性是上下文相关的，同样的词或者句子在不同的上下文情景下重要性有所不同的观点，通过构造了两层不同级别的注意力机制同时学习自然语言文档序列中的单词与句子的重要性，在文本分类任务上取得了很大程度的突破。由于契合本研究任务。其对于基于代码序列的多类型漏洞检测任务具有很好的可迁移性。

此外，本研究将上一模块生成的最终表示看作是单通道的类图像表示，选取网络结构相对简单，涉及参数少，计算量小，训练速度快且在文本分类任务中效果良好的 Text-CNN 神经网络模型实现进一步特征抽取。另外，其卷积操作的一维性良好契合了基于序列的漏洞检测任务，增强了方案的合理性与可解释性。最后，连接全连接层与 *softmax* 函数实现多类型漏洞的分类预测。

鉴于以上因素，本文首次提出了综合多层注意力网络（HAN）和多头文本卷积网络（Text-CNN）二者的漏洞检测方案。其整体框架分为 4 个部分。（一）输入层：使用上述介绍的主流词向量模型将预处理后的代码切片序列变成词向量。（二）隐藏层：隐藏层的第一部分为 HAN 网络结构，此网络一三级为双向神经网络，意在从前后两个方向提取文本上下文信息，二四级为两个注意力机制层，意在通过计算两个层级上的特征权重值，捕捉关键特征，最终输出对应的代码切片序列的特征向量；第二部分为 Text-CNN 网络结构，主要涉及到单向多头的卷积与池化操作，最后输出。（三）漏洞检测层：由全连接层接输出层组成，意在实现多类型漏洞检测。漏洞检测模型整体框架如图 2-1 所示。

以下，将对各模块的原理进行详细性阐述。

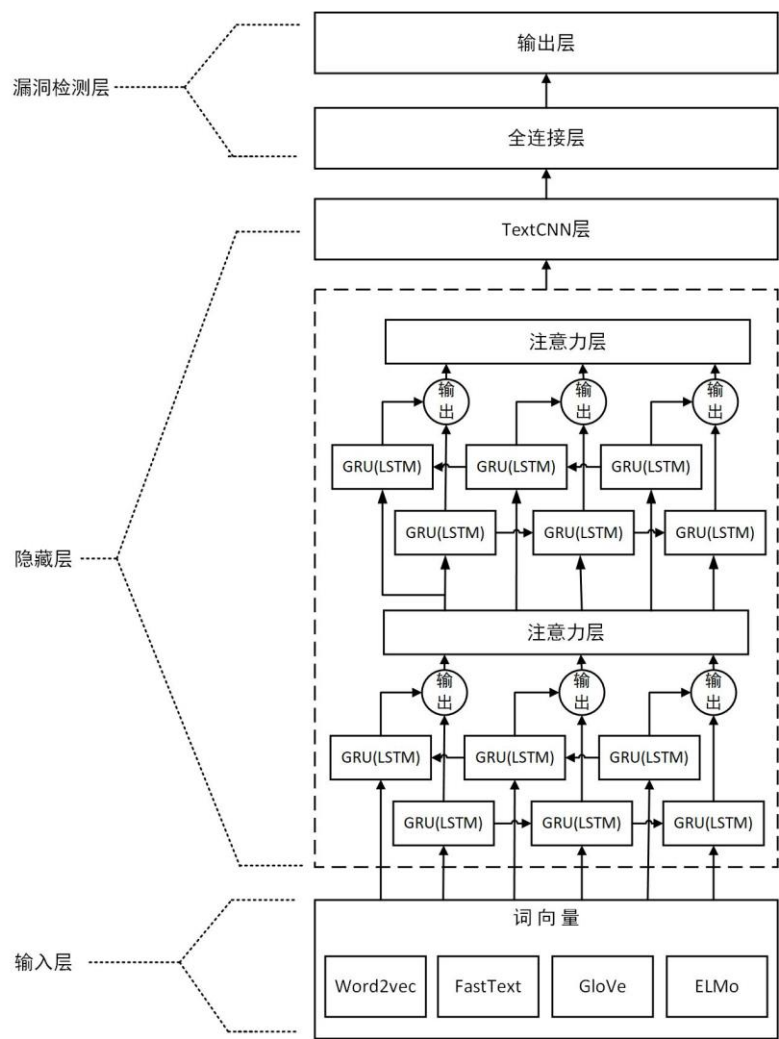


图 2-1 漏洞检测模型整体框架

Fig2-1 Overall framework of vulnerability detection model

2.1 输入层

本层的核心功能是通过词向量技术捉程序代码语句之间的语义关系并将其转换成神经网络能够识别的向量表示。优秀的词向量模型会使语义相似的词在对应维度的向量空间上有着相近的数学距离，即近义词会有相近的向量表示。现如今，词向量技术已由基础统计学范畴发展到基于神经网络的语言模型，充分借助神经网络的灵活性特征，进行复杂文本的向量表征。接下来，将对本方案所使用的词向量模型原理进行详细介绍。

2.1.1 Word2vec

Word2vec 为三层浅层神经网络：输入层、隐藏层与输出层。其中，隐藏层不包含激活函数，输入层维度与输出层维度相等。为获取词向量，需要用语料库对神经网络进行训练，最终学习到的隐层的权重作为一个查找表，用于获取对应词的词向量。其具体分为 Skip-Gram 和 CBOW 两种模型，二者首先通过设定窗口来划定每个单词作为中心词的上下文单词，如窗口值为 5，则前面五个单词与后面五个单词共同规定为中心词的上下文单词。Skip-Gram 的学习任务是利用中心词对上下文单词进行预测，而 CBOW 的学习任务则是利用上下文单词对中心词进行预测。其结构图见图 2-2。

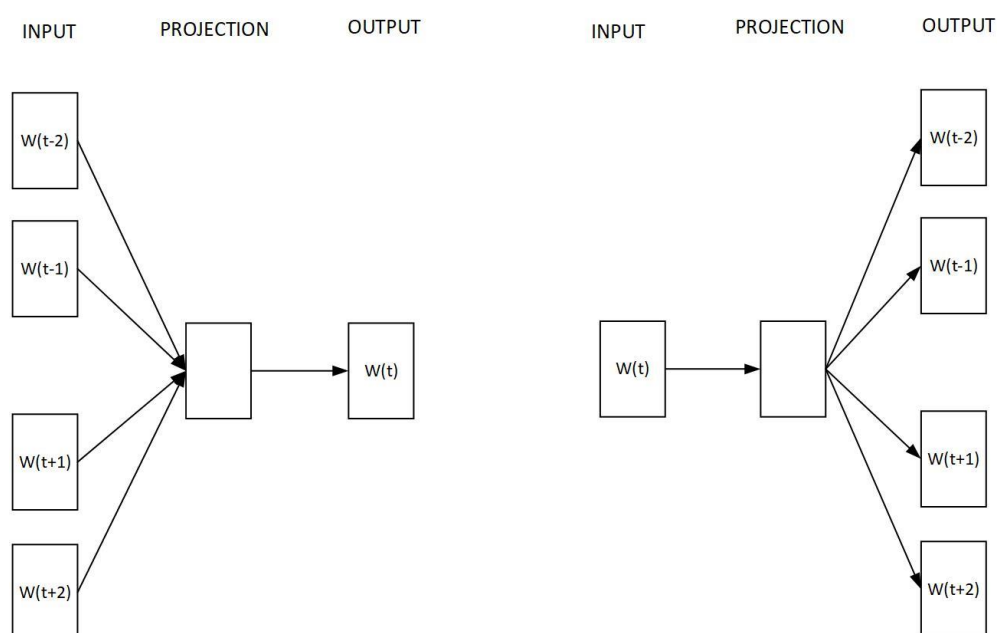


图 2-2 CBOW 与 Skip-Gram 结构图

Fig2-2 Structure diagram of CBOW and Skip-Gram

2.1.1.1 CBOW

CBOW 通过上下文单词来预测中心词，更适用于小型数据集，其具体流程如下：

设窗口大小为 m ，则上下文单词的个数为 $2m$ ，词向量维度为 D ，词库中单词个数为 $|V|$ ，

(1) 输入层：输入层为 $2m$ 个词的独热编码表示的结果,即输入为 $2m \times |V|$;

(2) 隐藏层：这里隐藏层主要是通过输入层的独热编码结果，来查找对应词的词向量，通过独热编码乘上词嵌入矩阵 $W_1: |V|*D$ ， $2m$ 个词对应的词向量的个数为 $2m$ 个，将这个 $2m$ 个词向量的结果相加求平均，得到的输出为 $1*|D|$ ；

(3) 输出层：由隐藏层输出的 $1*|D|$ 乘上输出权重矩阵 $W_2: |D|*|V|$ ，得到输出 $1*|V|$ ；

(4) 输出结果：得到的 $1*|V|$ 经过 softmax 转换成中心词取每个单词的概率 $1*|V|$ ；

(5) Loss：将预测取每个单词的概率作为 pred，真实结果为 true，以交叉熵为损失函数计算损失；

(6) 反向传播：以 Loss 为标准，不断反向传播更新 W_1 和 W_2 ，其中 W_1 就是需要的词向量的表示；

2.1.1.2 Skip-Gram

Skip-Gram 通过中心词来预测上下文单词，更适用于大规模数据集，其具体流程如下：

设窗口大小为 m ，则上下文单词的个数为 $2m$ ，词向量维度为 D ，词库中单词个数为 $|V|$ ，

(1) 输入层：输入为中心词的独热编码 $1*|V|$ ；

(2) 隐藏层：输入的中心词独热编码乘上隐层的词嵌入矩阵 $W_1: |V|*|D|$ ，得到隐层的输出 $1*D$ （也即为中心词对应的词向量）；

(3) 输出层：将隐层的输出 $1*D$ 乘上输出权重矩阵 $W_2: |D|*|V|$ 得到大小为 $1*|V|$ 的输出；

(4) 输出结果：得到的 $1*|V|$ 经过 softmax 转换成中心词取每个单词的概率 $1*|V|$ ；

(5) Loss：将预测取每个单词的概率作为 pred，真实结果为 true，以交叉熵为损失函数计算损失；

(6) 反向传播：以 Loss 为标准，不断反向传播更新 W_1 和 W_2 ，其中 W_1 就是需要的词向量的表示；

2.1.2 FastText

FastText 提出用于文本分类任务与词向量的获取，这里主要进行词向量生成相关的原理介绍。FastText 词向量模型是 Word2vec 的 Skip-Gram 模型的扩展，在之前模型的基础上引入对单词内部结构的考虑，通过拆分单词，将 word 转成 character n-grams 的形式，并将规定的约束符号<和>加在单词前后用以跟其他字符序列区分开。以单词 which 为例，取 character n-grams 中的 n 为 3，则其可拆分为<wh,whi,hic,ich,ch>，因此模型要学习的不仅是单词 which 本身，还要学习拆分后 wh,whi,hic,ich,ch 的向量表示，最终通过求和得到完整单词 which 的向量表示，即<wh,whi,hic,ich,ch>+<which>。相较于 Skip-Gram 模型，其允许跨单词共享表示，解决了罕见单词以及样本外单词的可靠表示问题。

2.1.3 GloVe

GloVe 模型针对以往词向量模型根据设定的窗口值的大小，仅考虑样本文本的局部信息而忽略整个文档语料的全局信息的问题而提出，其具体原理如下。

(1) 共现矩阵。所谓共现，指的是单词 i 出现在单词 j 的环境（以单词 j 为中心的左右 n 个单词区间，n 可根据任务需求设定）中。共现矩阵是基于整个语料库的单词对共现次数的统计表，矩阵中元素 x_{ij} 为表示单词 j 出现在单词 i 环境中的次数，即共现次数。共现矩阵表达语料库所有词之间关系，是整个文本语料库的全局统计特征。其生成步骤如下：

①首先构建一个大小为 $V \times V$ 空矩阵，即词汇表×词汇表，矩阵每个元素的初始值全部赋 0。矩阵中的元素坐标为 (i, j) 。②设定一个滑动窗口的大小(例如取值为 m)，则窗口内的前后总计 $2m-1$ 个单词纳入共现次数的统计。其中，在语料库的首位位置会出现空窗口的情况，此时，不计入统计。③从语料库的第一个单词作为中心词 i，以 1 的步长滑动该窗口，统计词与词之间的共现次数，并将对应的次数依次累计到矩阵 (i, j) 位置上。④不断滑动窗口直至走完所有语料库中的单词，最终得到一个基于对应语料库的完整的共现矩阵。

(2) 共现概率。现得到共现矩阵 X ，元素 x_{ij} 为单词 i 出现在单词 j 环境中的次数，令 $x_i = \sum_k x_{ik}$ 即将共现矩阵的第 i 行求和，含义为所有词出现在单词 i 环境中的次数，则定义，

$$P_{ij} = P(j|i) = \frac{x_{ij}}{x_i} \quad (2-1)$$

为单词 j 出现在单词 i 环境中的概率，即单词 i 与单词 j 的共现概率。

(3) 共现概率比。指的是单词共现概率的比值,定义为，

$$ratio_{i,j,k} = \frac{P_{ik}}{P_{jk}} \quad (2-2)$$

共现概率的比值能够很好的反应不同词语之间的关联特性，其关系如表 2-1 所示。

表 2-1 共现概率比关系表

Tab2-1 Co-occurrence probability ratio relationship table

$ratio_{i,j,k}$ 的值	单词 j , k 相关	单词 j , k 不相关
单词 i , k 相关	趋近 1	很大
单词 i , k 不相关	很小	趋近 1

(4) 设计词向量函数。通过合理设计词向量的计算函数去表达共现概率比，使得词向量与共现矩阵保持一致性，就能使模型在构建词向量时充分考虑语料库的全局特征。作者最终设计的损失函数为，

$$J = \sum_{i,j}^N f(X_{ij})(v_i^T v_j + b_i + b_j - \log(X_{ij}))^2 \quad (2-3)$$

其中，权重函数为，

$$f(x) = \begin{cases} \left(\frac{x}{x_{max}}\right)^\alpha, & x < x_{max} \\ 1, & x \geq x_{max} \end{cases} \quad (2-4)$$

最后，对于 GloVe 模型，其流程如下：1.采集训练数据。训练数据需要满足共现矩阵对应位置不为 0。2.随机初始化采集样本的词向量值。3.随机初始化以上两个偏置的值。4.内积计算与平移操作。5.计算与 $\log(X_{ij})$ 的损失值。6.计算梯度数值。7.反向传播更新参数值（词向量与偏置），使得词向量的内积加偏置不断趋近于共现次数的对数。多次循环以上 7 个过程直到结束条件。

2.1.4ELMo

语言嵌入表示模型，即 ELMo。解决的新问题是现代文本中极为普遍的一词多义现象，即对于相同的单词，在不同的上下文环境中可以有着不同的意义，代码语言也具有相同的规律，相同的参数名称在不同的函数中也有着截然不同的意义，然而，无论是 Word2vec、FastText 还是 GloVe 模型，一个单词仅与一个词向量一一对应，无法反应这种多义关系，针对多义词建模的 ELMo 模型应运而生，其是采用双层双向长短期记忆网络（Bi-LSTM）的预训练模型，首先基于大规模语料库进行预训练，得到初始词向量，之后按照下游任务的需求不同，对初始词向量做进一步的调整，即在预训练的基础上可根据实际任务进行词向量定制，其结构图如图 2-3 所示。

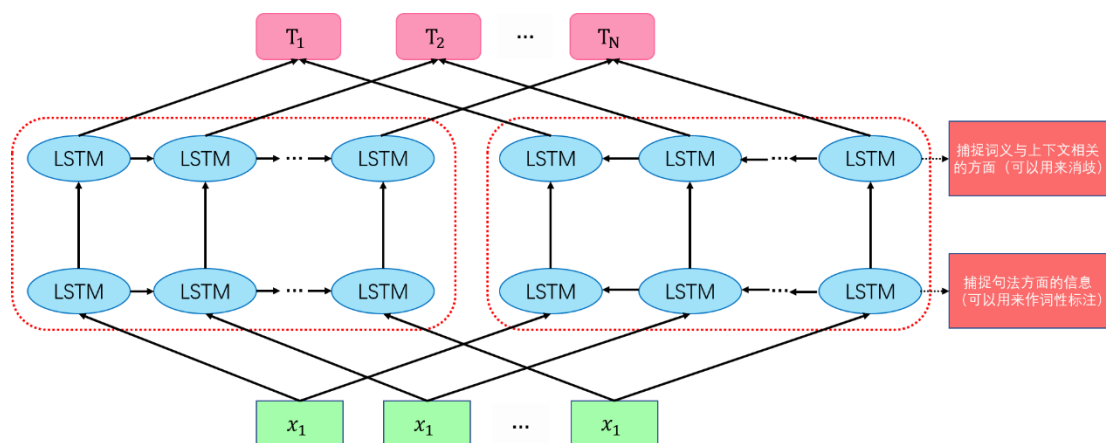


图 2-3 ELMo 模型结构图

Fig2-3 ELMo model structure diagram

ELMo 首先需要借助公开的大型数据集上进行模型的预训练，后根据下游任务的数据集的差异，基于子任务数据集对每个词向量的数值进行调整，最后得到适配子任务的完备词向量。对于给定句子 $T = [t(1), t(2), \dots, t(N)]$ ，通过优化目标函数

$$\sum_{k=1}^N \left(\log p(t_k | t_1, \dots, t_{k-1}; \theta_x, \vec{\theta}_{LSTM}, \theta_s) \right) + \left(\log p(t_k | t_{k+1}, \dots, t_N; \theta_x, \vec{\theta}_{LSTM}, \theta_s) \right)$$

其中， θ_x 表示单词输入时的词向量， θ_s 是 softmax 层的参数， $\vec{\theta}_{LSTM}$ 表示前向 LSTM 的参数， $\tilde{\theta}_{LSTM}$ 表示后向 LSTM 的参数，进行词向量的计算，流程如下：

- ①从预训练得到的静态的词向量表里查找单词的词向量 $V(1), \dots, V(N)$ 用于输入。
- ②将单词词向量 $V(1), \dots, V(N)$ 分别输入第一层的双向长短期记忆网络，分别得到对

应的前向输出 $\vec{h}_{1,1}^{LM}, \dots, \vec{h}_{N,1}^{LM}$ ，和后向输出 $\overleftarrow{h}_{1,1}^{LM}, \dots, \overleftarrow{h}_{N,1}^{LM}$ 。

③将前向输出 $\vec{h}_{1,1}^{LM}, \dots, \vec{h}_{N,1}^{LM}$ 与后向输出 $\overleftarrow{h}_{1,1}^{LM}, \dots, \overleftarrow{h}_{N,1}^{LM}$ ，分别作为第二层长短期记忆网络的输入传入，依次得到第二层前向输出 $\vec{h}_{1,2}^{LM}, \dots, \vec{h}_{N,2}^{LM}$ 与第二层的后向输出 $\overleftarrow{h}_{1,2}^{LM}, \dots, \overleftarrow{h}_{N,2}^{LM}$ 。

④则单词 i 最终可以得到的词向量包括 $V(i), \vec{h}_{N,1}^{LM}, \overleftarrow{h}_{N,1}^{LM}, \vec{h}_{N,2}^{LM}, \overleftarrow{h}_{N,2}^{LM}$ ，如果采用 M 层的 biLSTM 网络则最终可以得到 $2M+1$ 个词向量。

⑤采用如下运算将所有词向量进行加权融合得到最终词向量，

$$ELMo_i^{task} = \gamma^{task} \sum_{j=0}^L s_j^{task} h_{i,j}^{LM} \quad (2-5)$$

其中， γ^{task} 是一个与下游任务相关的系数，其作用是允许根据任务的不同缩放 ELMo 向量， s_j^{task} 是使用 softmax 归一化的权重系数。

2.2 多层注意力网络（HAN）

多层注意力网络（HAN）是针对文本分类任务提出的一种新的神经网络架构，其利用人类阅读时下意识对文章梳理层次抓取重点内容的习惯，用机器进行模拟，首次提出了层次化的编码层结构，并将两层注意力机制引入到模型当中。其具体的特点如下：（1）将文本划分为“词-句子-文章”的三层结构，并按照此结构依次进行特征表示，首先以词为输入构建对应的句子表示，然后将所有的句子表示其聚合成文本表示。（2）将词语层次（word-level）与句子层次（sentence level）的双层次的注意力机制引入模型，进而赋予了模型能够按照“词语-句子”两个层次分别给予不同“注意力”的能力。其在多个文本分类数据集上取得了远超之前模型的表现，鉴于基于代码切片序列的漏洞检测与文本分类任务在原理上具有一定程度的相似性，考虑将其引入作为漏洞检测模型的基本单元网络之一。

2.2.1 网络结构

HAN 由一个词序列编码器、一个词级注意力层、一个句编码器和一个句级注意力层四部分所构成。其网络整体架构参照图 2-4 所示。

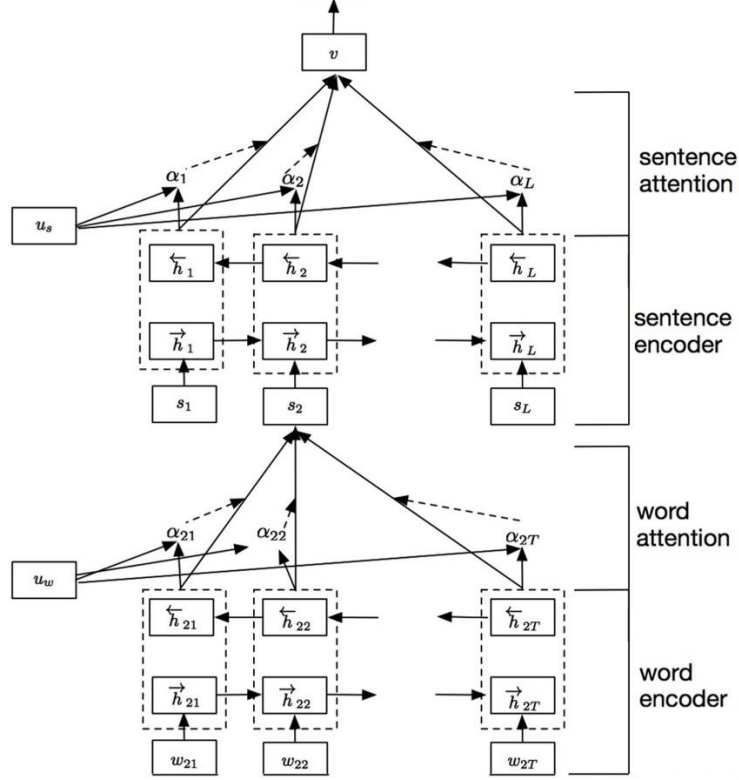


图 2-4 HAN 模型架构

Fig2-4 Model architecture of HAN network

2.2.2 基本原理

(1) 词序列编码器：此层的核心网络架构是双向 GRU 网络。针对一个由词序列 ω it, $t \in [0, T]$ 组成的句子，首先利用词向量矩阵 W_e 把词转化为词向量双向 GRU 能够汇总单词来自两个方向的上下文信息，进而获得此层隐藏层的输出。该层公式表示如下。

$$x_{it} = W_e \omega_{it}, t \in [0, T] \quad (2-6)$$

$$\vec{h}_{it} = \overrightarrow{GRU}(x_{it}), t \in [0, T] \quad (2-7)$$

$$\overleftarrow{h}_{it} = \overleftarrow{GRU}(x_{it}), t \in [0, T] \quad (2-8)$$

值得注意的是，该层一定的灵活性，核心网络架构可以更换成双向长短期记忆网络（Bi-LSTM），如此，该层的公式表示如下。

$$x_{it} = W_e \omega_{it}, t \in [0, T] \quad (2-9)$$

$$\vec{h}_{it} = \overrightarrow{LSTM}(x_{it}), t \in [0, T] \quad (2-10)$$

$$\overleftarrow{h}_{it} = \overleftarrow{LSTM}(x_{it}), t \in [0, T] \quad (2-11)$$

其中， $\vec{h}_{it}$ 与 $\overleftarrow{h}_{it}$ 叫做给定单词 ω_{it} 的前向隐藏态与反向隐藏态，通过连接二者 $h_{it} = [\vec{h}_{it}, \overleftarrow{h}_{it}]$ 获得单词的表示，其对单词 ω_{it} 为中心的句子信息进行了整合，包含周围两个方向的信息。

（2）词级注意力层：将 h_{it} 通过 MLP 提供单词表示以获取作为 h_{it} 的隐藏表示 μ_{it} ，然后通过计算与单词级别上下文向量 μ_{ω} 的相似性来测量单词作为 μ_{it} 的重要性，经过softmax操作获得了一个归一化的注意力权重矩阵 α_{it} ，代表句子 i 中第 t 个词的权重。之后，我们计算基于权重的单词表示的加权来获得句子向量 s_i 。这里的上下文向量 μ_{ω} 是在训练网络的过程中学习获得的。该层公式表示如下。

$$\mu_{it} = \tanh(W_{\omega} h_{it} + b_{\omega}) \quad (2-12)$$

$$\alpha_{it} = \frac{\exp(\mu_{it}^T \mu_{\omega})}{\sum_t \exp(\mu_{it}^T \mu_{\omega})} \quad (2-13)$$

$$s_i = \sum_t \alpha_{it} h_{it} \quad (2-14)$$

（3）句编码器：此层的核心网络架构是双向 GRU 网络。其原理类似于词序列编码器，对于给定的句子 s_i ，可得到相应的句子表示 h_i ，其同样包含句子周围两个方向的信息。该层公式表示如下。

$$\vec{h}_i = \overrightarrow{GRU}(s_i), i \in [1, L] \quad (2-15)$$

$$\overleftarrow{h}_i = \overleftarrow{GRU}(s_i), i \in [L, 1] \quad (2-16)$$

同上，核心架构更换为 Bi-LSTM 后公式表示如下。

$$\vec{h}_i = \overrightarrow{LSTM}(s_i), i \in [1, L] \quad (2-17)$$

$$\overleftarrow{h}_i = \overleftarrow{LSTM}(s_i), i \in [L, 1] \quad (2-18)$$

(4) 句级注意力层：该层原理与第(2)层十分类似，利用句子级别的上下文向量 μ_s 来衡量一个句子在整篇文档中的重要性，最终输出的 v 向量作为文本向量，整合了文本中句子的所有信息，将其作为初始文本的高级表示，用作最终的分类任务。该层公式表示如下。

$$\mu_i = \tanh(W_s h_i + b_s) \quad (2-19)$$

$$\alpha_i = \frac{\exp(\mu_i^T \mu_s)}{\sum_i \exp(\mu_i^T \mu_s)} \quad (2-20)$$

$$v = \sum_i \alpha_i h_i \quad (2-21)$$

2.3 文本卷积神经网络 (Text-CNN)

文本卷积神经网络 (Text-CNN) 是针对文本分类任务改进的卷积神经网络架构，在传统卷积神经网络的基础上，根据人对文本的阅读顺序的一维特性，对输入层进行改良，使得卷积核只在一维方向滑动提取特征。Text-CNN 的网络结构相对简单，涉及到的参数数目少，计算量少，训练速度快，其与现如今流行的词向量嵌入方法结合，在文本分类领域有着非常不错的表现。鉴于基于代码切片序列的漏洞检测与文本分类任务在原理上具有一定程度的相似性，考虑将其引入作为漏洞检测模型的基本单元网络之一。

2.3.1 网络结构

Text-CNN 模型由以下几部分组成：输入层、卷积层、池化层。其整体架构参照图 2-5 所示。

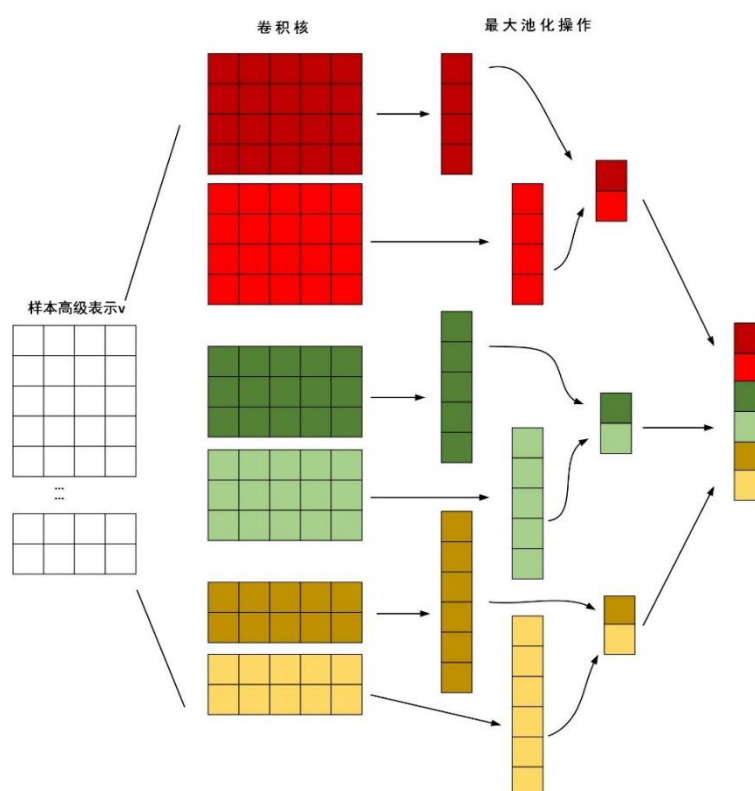


图 2-5 Text-CNN 模型架构

Fig2-5 Model architecture of Text-CNN network

2.3.2 基本原理

(1) 输入层：该层的主要作用是将上层网络结构得到的漏洞样本的文本表示 v 作为输入。

(2) 卷积层：该层用于提取单词特征，核心操作为卷积计算。区别于图像处理领域，为保证卷积操作在方向上的一维特性，Text-CNN 模型卷积核的列宽需要等同于词向量维度，且一般采用多卷积核进行多次计算的策略。实验表明，在文本分类领域，卷积核的多通道设置并没有明显提升模型的分类能力，因此，常采用单通道设置来简化运算。

(3) 池化层：该层的输入是卷积层的输出，目前存在两种池化计算策略：一是对每个通道的所有数值求取均值，叫做平均池化；二是求取每个通道数值的最大值，叫做最大值池化。最终将池化得到的数值进行拼接后作为该层的输出。

2.4 漏洞检测层

漏洞检测层以上层的输出作为本层的输入，意在展现最终的漏洞分类结果。漏洞检测层的核心部分是全连接层与输出层。其中，全连接层首先接收输入后接 $softmax$ 激活函数进行归一化处理，将模型输出值转换为输出概率，输出每个漏洞类型标签的预测概率。之后，输出层以数字标记的形式输出漏洞检测最终的检测结果。

2.5 实现流程

2.5.1 词向量生成

本阶段的主要任务是将程序代码转化成向量模式，输入漏洞检测模型。本研究采用了四种不同的词向量模型进行词向量转化，操作流程为：首先，将标准化后的数据集程序代码看作文本，以此为基础建立语料库，使用四种词向量技术分别得到每个模型对应的单词与词向量的映射关系字典。其次，将每个程序代码包含的单词的词向量合并拼接成矩阵形式作为该样本的向量表示。

由于每个数据样本包含的单词的个数不相同，上述操作最终得到的向量矩阵的维度也存在差异，因此需要对向量矩阵的格式进行规范，本研究采用的规范策略如下：（1）设定向量矩阵的最大维度阈值 t ；（2）对于维度超过 t 的数据样本，随机抽取其中 t 维度作为对应向量矩阵；（3）对于维度不超过 t 的数据样本，采用末尾补“0”操作，使得维度等于 t 。阈值 t 将作为超参数，按照神经网络的调参策略进行具体值的确定。

整个过程的流程图如图 2-6 所示。

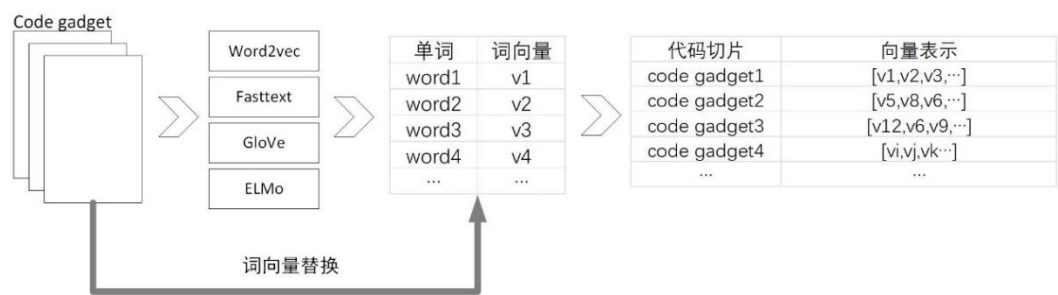


图 2-6 词向量生成流程图

Fig2-6 Word vector generation flowchart

2.5.2 训练与检测

在训练阶段，模型接受词向量输入，学习漏洞特征，详细流程如下：首先，将数据样本的词向量矩阵作为隐藏层的输入。其次，确定漏洞检测模型的最优超参数，本研究首先给出了包括优化器、学习率、批量数值、隐层节点数、向量矩阵维度阈值 t 等需要调整的超参数列表，后采用网格搜索与交叉验证相结合的方法获得以上超参数的最优值集合，以此获取一个具有较优性能的漏洞检测模型。最终，经过训练得到一个参数完备的漏洞检测模型。实验参数如表 2-2 所示，在训练中采用 ReduceLROnPlateau 参数，当模型的性能不在提升的时候，减小学习率。

表 2-2 模型参数

Tab2-2 Model parameter		
参数	含义	值
Lr	初始学习率	0.001
Batch_size	每个批次训练样本的数量	128
Embed_size	词向量维度	100
Num_hiddens	隐藏层特征数量	128
Epoch	训练轮次	80
Dropout	减少过拟合	0.5
SEQUENCE_LENGTH	最大序列长度 t	50

在测试阶段，使用词向量模型生成测试集数据样本对应的词向量矩阵输入隐藏层进行分类。若被分类为“0”，则待测样本被认定为无漏洞。若被分类的标记大于“0”，则待测样本将被认定为存在漏洞，并根据标签与漏洞类型的映射表格，报告漏洞对应的具体类型。整个阶段的流程图如图 2-7 所示。

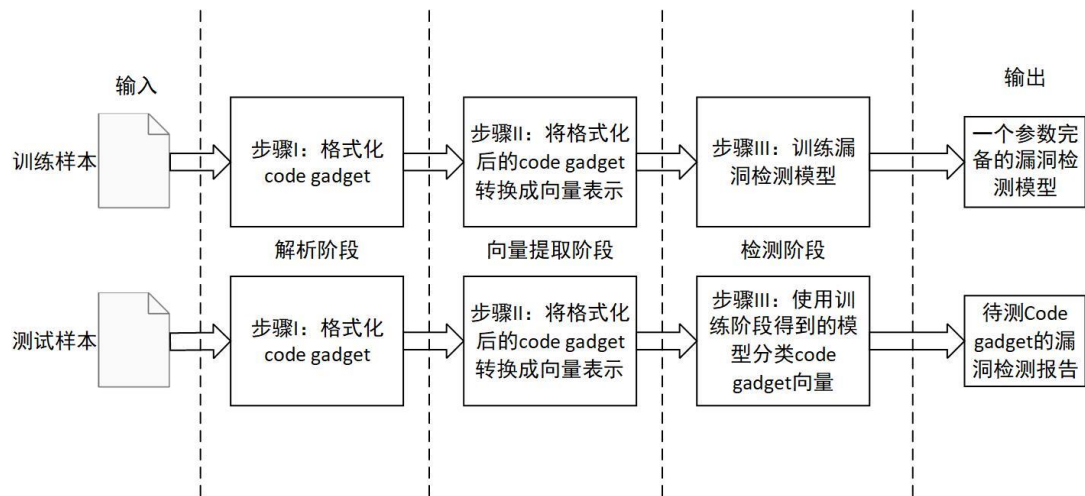


图 2-7 训练与检测流程图

Fig2-7 Training and testing flowchart

2.6 本章小结

本章首先给出了多类型漏洞检测模型的整体框架，由输入层、隐藏层及漏洞检测层三大基本单元构成，之后对每层的结构以及原理做了详细介绍。输入层由 4 种词向量模型组成，用于研究不同词向量模型对漏洞检测模型的性能的影响，是本文研究的关键问题之一。隐藏层由两种类型的网络结构前后连接而成，分别是 HAN 网络与 Text-CNN 网络，是该漏洞检测模型的核心部分。漏洞检测层以全连接层接收隐层的输出后接 softmax 分类器输出，用于多类型漏洞检测。最后，详细说明了词向量生成的流程、策略以及漏洞检测模型的训练与测试流程、超参数设定策略与模型参数列表。

第三章 基于图与序列的函数级源代码漏洞检测方案的设计

该漏洞检测方案将函数级源程序样本作为输入，包括以下两大模块：特征抽取与特征学习。特征抽取阶段，使用两种方式对源代码进行处理：1.抽取函数源代码对应的多类型边结构图。2.抽取函数源代码对应标记序列。特征学习阶段，分别生成以上两种形式的中间表示对应的实值向量，并将二者集成拼接。最后，使用全连接网络输出漏洞检测结果。漏洞检测模型的整体框架如图 3-1 所示。

以下，将对各模块的原理进行详细性阐述。

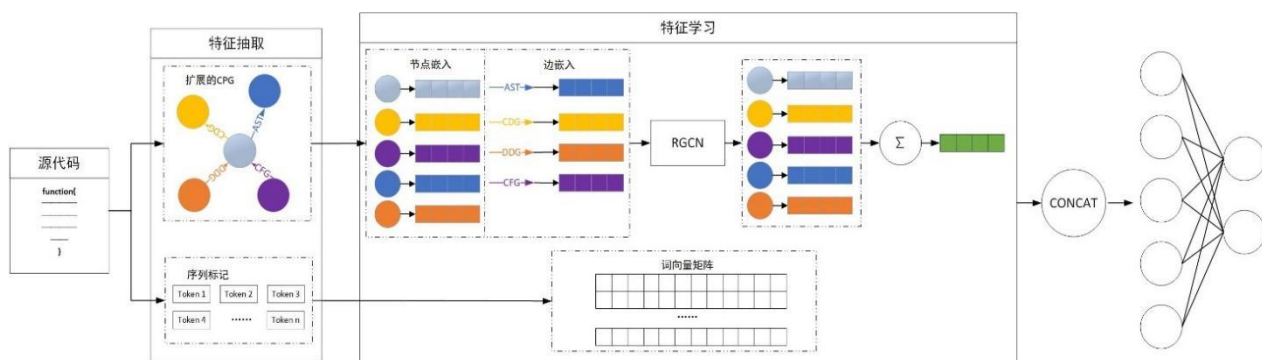


图 3-1 本研究漏洞检测方案整体架构

Fig3-1 The overall architecture of the vulnerability detection scheme in this study

3.1 基于图的特征学习

3.1.1 图的生成

为学习源代码中的结构信息并提取漏洞特征，需要将其转换成合适的结构图。结构图的质量在很大程度上决定了从训练数据中学习到的特征数量，进而影响模型的漏洞检测性能。代码属性图(简称 CPG)^[54] 提供了控制流图(简称 CFG)和数据流图(简称 DFG)中的元素以及抽象语法树(简称 AST)和程序依赖图(简称 PDG)的代码组合，是现存最综合全面的程序图表示，本文选择其作为源代码的初始结构图。

源代码函数对应的 CPG 按照以下流程生成：1.解析源代码函数生成 AST。2.应用控制流分析生成 CFG。3.应用依赖性分析生成 PDG。4.在 AST 中添加 CFG 与 PDG 边生成 CPG。以上过程使用工具 Joern 实现。

CPG在富含多种结构信息的同时,缺失了语句中的顺序信息,为解决此问题,本文在初始结构图 CPG 基础上增加自然代码序列边(NCS)^[55]对其进行了扩展,通过在AST 中的相邻叶节点之间从左到右添加有向边,增加了代码编辑的时序信息,辅以漏洞检测。本文将最终得到的结构图记为 $CPG' = \langle V, E \rangle$, 其中, $V = \{v | v \in ASTnode\}$ 表示图中的节点集合, $ASTnode$ 表示函数的 AST 中的节点。每个节点包含节点类型与节点代码两种元素。 $E = \{e | e \in CFGedge \text{ 或 } e \in DDGedge \text{ 或 } e \in CDGedge \text{ 或 } e \in ASTedge \text{ 或 } e \in NCSedge\}$ 表示图中的边集合, 这些边分别反映了节点之间的控制流、数据依赖、控制依赖、语法结构与时序信息。图 3-2 对应一个缓冲区溢出漏洞的简单样例, 其 main 函数对应的 CPG'如图 3-3 所示。

后续任务中基于 CPG'生成神经网络能处理的实值向量至关重要, 本文将综合考虑结构图中的节点及边。

```

1  int main(int argc, char **argv) {
2      char buf[264];
3      if (argc>1){
4          strcpy(buf, argv[1]);
5      }
6  }
    
```

图 3-2 缓冲区溢出漏洞代码样例

Fig3-2 Code sample of Buffer overflow vulnerability

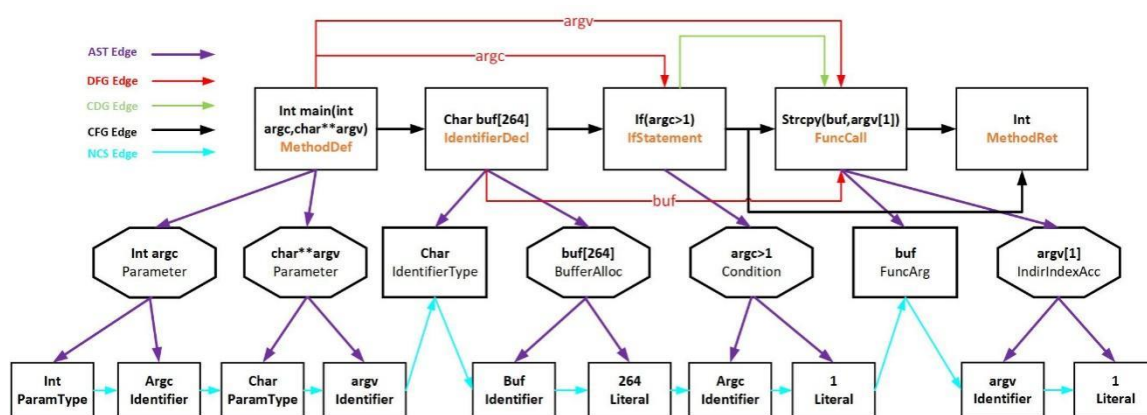


图 3-3 CPG'

Fig3-3 CPG'

3.1.2 节点嵌入

本模块分别对节点类型与节点代码采用不同嵌入方法生成初始向量：1.对于节点类型，采用独热编码(one-hot)生成对应向量 x_i ；2.对于节点代码，首先，完成标准化处理，将源代码中用户自定义的变量名称与函数名称采用统一标准进行重命名，用 VAR 与序号的组合替代自定义变量名，用 FUN 与序号的组合替代自定义函数名，如“VAR1”和“FUN1”，减少语料库规模与噪声干扰，提升向量生成的质量；其次，使用基于编程语言的预训练模型 CodeBert^[57]训练的单词嵌入权重初始化嵌入层权重，CodeBert 模型使用的字节对编码（BPE）标记器^[56]在很大程度上缓解 Out-Of-Vocabulary（OOV）问题；最后，连接双向长短期记忆网络(Bi-LSTM)聚合局部语义信息，获得最终向量表示 y_i 。

最终，连接 x_i ， y_i 得到节点 v_i 对应的向量表示 g_i 。

上述过程的形式化表述如下，

对于节点类型 $v_{[type,i]}$ ：

$$x_i = one-hot(v_{[type,i]}) \quad (3-1)$$

对于源代码 $S=\{s_1, s_2, s_3, \dots, s_n\}$ ，其中， s_i 对应节点 v_i 中的代码。首先，进行标准化处理：

$$S' = normalization(S) \quad (3-2)$$

得到 $S'=\{s'_1, s'_2, s'_3, \dots, s'_n\}$ ，其中， s'_i 对应 CPG'中的节点 v_i 标准化后的源代码。

其次，通过 CodeBert 预训练的 BPE 标记器进行标记并使用 CodeBert 嵌入层权重来初始化自嵌入权重，以获得每个标记的向量表示 $e_{ij} \in E_i = \{e_{i1}, e_{i2}, e_{i3}, \dots, e_{in}\} : :$

$$tokens_i = BPE(s_i) \quad (3-3)$$

$$E_i = Codebert(tokens_i) \quad (3-4)$$

最后，连接 Bi-LSTM 获取与节点 v_i 对应的语句 s_i 的向量表示 y_i ：

$$\vec{h}_v, \overleftarrow{h}_l = Bi-LSTM(E_i) \quad (3-5)$$

$$y_i = \text{Sum}(\vec{h}_i, \overleftarrow{h}_i) \quad (3-6)$$

其中, $\vec{h}_i, \overleftarrow{h}_i$ 是 Bi-LSTM 的最终输出。

最终, 连接 x_i, y_i 得到节点 v_i 的最终向量表示 g_i :

$$g_i = \text{CONCAT}(x_i, y_i) \quad (3-7)$$

3.1.3 边嵌入

本模块分别对边方向与边类型采用不同嵌入方法生成初始向量: 1.对于边方向, 采用深度优先遍历(DFS)将节点排序并从 0 开始编号, 以节点编号数对的形式编码边方向; 2.对于边类型, 采用标签编码(label encoding)将有限边类型进行编码。

上述过程的形式化表述如下,

函数源代码对应结构图图记为 $\text{CPG}' = \langle V, E \rangle$, 其中, V 代表节点集合, E 代表边集合。

对于边方向, 有:

$$I_E = \{ \langle v_i.\text{order}, v_j.\text{order} \rangle \mid e = \langle v_i, v_j \rangle, e \in E \} \quad (3-8)$$

对于边类型, 有:

$$T_E = \{t_e \mid e \in E\} \quad (3-9)$$

$$t_e = \begin{cases} 4 & \text{if } e.\text{type} = \text{CDG} \\ 3 & \text{if } e.\text{type} = \text{DFG} \\ 2 & \text{if } e.\text{type} = \text{AST} \\ 1 & \text{if } e.\text{type} = \text{CFG} \\ 0 & \text{if } e.\text{type} = \text{NCS} \end{cases} \quad (3-10)$$

以下为举例说明, 图 3-4 表示一个函数源代码对应的边嵌入过程, 按照深度优先遍历算法对节点编号, 图中 5 条有向边分别对应 5 种不同边类型, 其边方向与边类型对应的向量形式如图所示。

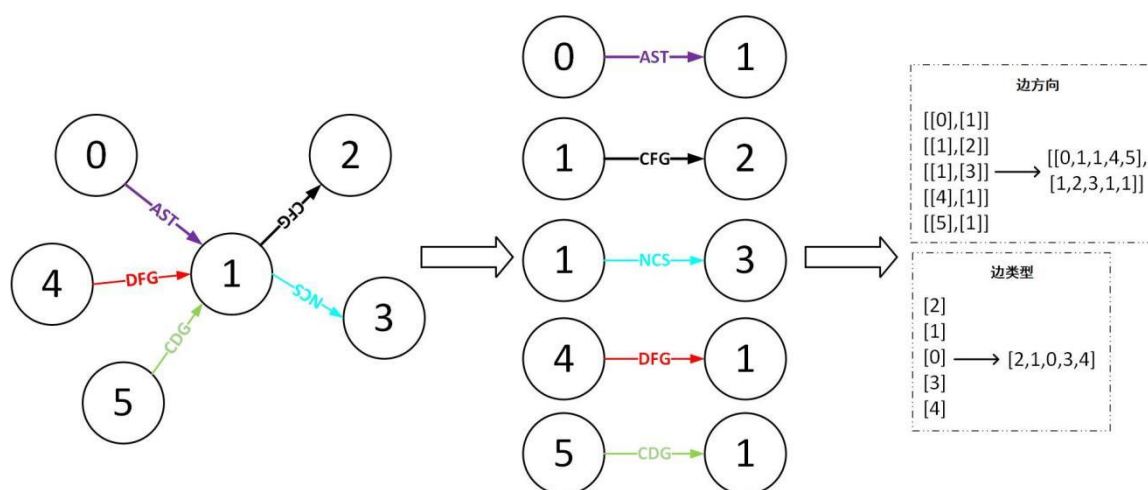


图 3-4 边嵌入策略实例说明

Fig3-4 Example Description of Edge Embedding Strategy

3.1.4 图神经网络

以上环节孤立考虑每个节点，仅根据单个节点包含的元素进行向量初始化，缺乏相邻节点的状态信息，无法较好反映源代码的结构信息。本研究采用关系图卷积网络（RGCN）完成邻域聚合，更新节点状态。

图卷积网络（GCN）是 CNN 在图结构上的扩展。在给定图上应用图卷积运算后，通过聚合其相邻节点的所有特征来更新节点。与连续计算节点状态的递归图神经网络（如 GGNN）相比，GCN 每层更新一次节点向量，因此效率更高，可扩展性更强。RGCN 在 GCN 的基础上进行了增量改进，其通过在每种关系类型下用相邻节点更新节点来区别对待不同类型的边，更适合处理本文所抽取的多类型边异构图 CPG'。如图 3-5 所示，在每个 RGCN 层中，更新单个节点需要按边类型积累其所有相邻节点的信息，这些信息随后与该节点本身的特征相结合，通过激活函数（本文采用 *ReLU*）传递，得到最终表示。

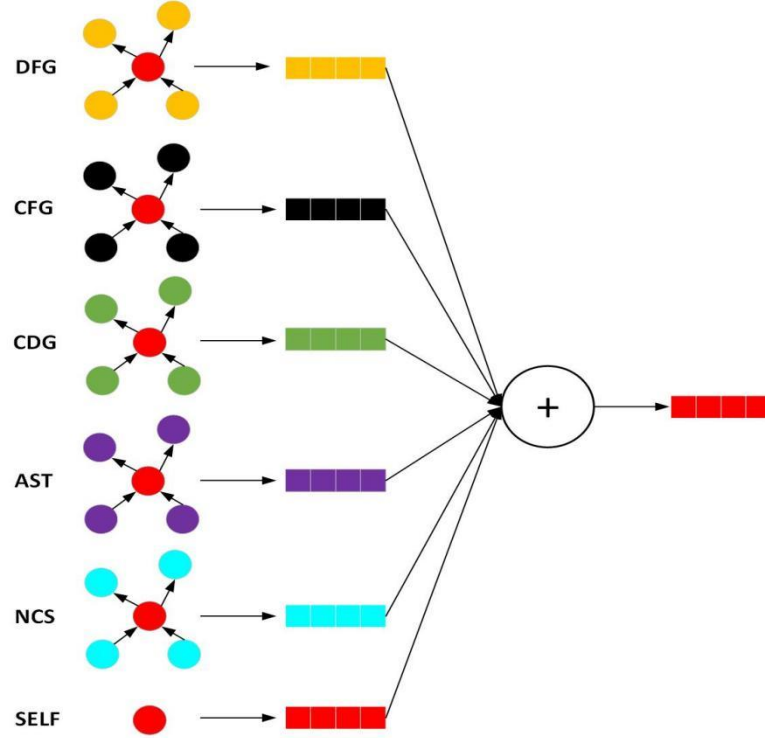


图 3-5 节点更新过程

Fig3-5 Node update process

上述过程的形式化表述如下，

函数源代码对应结构图记为 $CPG' = \langle V, E \rangle$ ，其中， V 代表节点集合， E 代表边集合。

单个节点向量将按照以下公式进行更新：

$$g_i^{(l+1)} = g_i^{(l)} + \sum_{r \in R} \sum_{v_j \in V_r^j} \frac{1}{V_r^j} \sigma(W_r [g_i^{(l)}, g_j^{(l)}, I_{i,j}, T_{i,j}] + b_r) \quad (3-11)$$

其中， $\sigma(\cdot)$ 代表非线性激活函数 $\text{ReLU}(\cdot)$ ， R 代表边的类型集合， r 代表对应的 5 中边关系之一， V_r^j 代表在当前关系 r 下与当前节点相邻的节点集合， $I_{i,j} \in I_E$ ， $T_{i,j} \in T_E$ 分别代表节点 v_j 与 v_k 之间的边方向与边类型， W_r 与 b_r 代表可学习的参数， $g_i^{(l+1)}$ 代表更新后节点的最终状态。

3.1.5 图嵌入

图嵌入模块为基于图的特征学习的最后一步，目标是得到最终的图向量表示。本文通过聚合所有更新后的节点向量，生成代表整个 CPG' 的单个向量，即：

$$G = \sum_{v_i \in V} g_i^{(l)} \quad (3-12)$$

3.2 基于序列的特征学习

基于序列的特征学习是本方案理解源代码全局语义信息的关键过程。现有研究普遍采用以 Word2vec、GloVe 及 FastText 为代表的词向量模型生成源代码对应的实值向量，上述模型无法基于不同的代码上下文为相同标记生成不同向量，即仅为相同标记生成唯一向量，不考虑标记对应变量值的变化，忽略了上下文信息在漏洞检测工作中的关键作用。

本研究采用基于代码语言的预训练模型 CodeBert^[57]来生成源代码的特征向量，充分考虑上下文信息。相较传统词向量模型，CodeBert 的模型结构要更复杂，其结构继承了 BERT 的架构，基于一个 12 层双向 transformer，每层包含 12 个自注意头，每个自注意头的大小设为 64，隐藏维度设为 768，transformer 的多头注意力机制能够集中数据流的多个关键变量，助于分析和跟踪潜在的漏洞代码数据，帮助学习需长期依赖性分析的漏洞代码模式。此外，其内置字节对编码（BPE）标记器极大程度的缓解了 OOV 问题，预训练技术也克服了数据集中漏洞样本过少或漏洞样本与非漏洞样本的不平衡所带来的过拟合问题。最后，对于每一个函数级源代码样本，CodeBert 编码器生成对应的向量矩阵，封装其语义含义。

上述过程的形式化表述如下，

f_i 代表一个函数级源代码样本输入，其向量矩阵 P_i 由以下公式获取：

$$P_i = \text{CodeBert}(f_i) \quad (3-13)$$

3.3 特征融合

以上流程过后，分别获得了所有函数级源代码样本 f_i 的基于结构图生成的向量表示 G 与基于序列生成的向量表示 P_i 。本研究采用连接操作组合二者，进而集成两种特征，生成所有源代码函数对应的最终向量表示 H_i ，公式如下：

$$H_i = \text{CONCAT}(G, P_i) \quad (3-14)$$

3.4 分类器

鉴于全连接神经网络（FCNN）的卓越性能和在顶级分类器中的主导地位，本文使用全连接网络作为漏洞检测分类器，其由输入层、隐藏层与输出层三部分组成。

上述特征融合模块的输出被作为输入层，最后，输出层将利用隐藏层特征进行漏洞预测。

上述过程的形式化表述如下，

函数源代码 f_i 对应最终向量表示 H_i ，其预测标签 $\hat{y}_i$ 可通过以下公式得出：

$$H_i' = \sigma(W_1 * H_i + b_1) \quad (3-15)$$

$$\hat{y}_i = \text{Sigmoid}(W_2 * H_i' + b_2) \quad (3-16)$$

其中， σ 代表隐层激活函数， W_1 、 W_2 、 b_1 与 b_2 代表可学习的参数。

3.5 本章小结

本章首先给出了漏洞检测模型的整体框架，由基于图的特征学习、基于序列的特征学习、特征融合及漏洞检测模块四大基本单元构成。之后对每层的结构以及原理做了详细介绍。在基于图的特征学习模块，一方面扩展了当前的图形式，提出了更加综合的图；另一方面创新了图节点与边的嵌入策略，借助关系图卷积网络生成最终图表示。在基于序列的特征学习模块，引入基于代码语言的预训练模型 CodeBert 生成序列向量表示。最后集成两种表示形式用以漏洞检测。

第四章 实验设计与结果分析

4.1 实验环境

两种漏洞检测方案的开发环境均是 Linux 操作系统，使用 Python 语言进行开发，深度学习库选择以 Tensorflow-gpu 为后端的 Keras 框架开发，其具体内置版本信息如表 4-1 所示：

表 4-1 实验环境参数表

Tab4-1 Table of Experimental Environment Parameters

参数	值
GPU 版本	NVIDIA GeForce GTX 2070 SUPER
内存大小	16GB
磁盘空间	1TB
环境配置	Keras2.2.5、Tensorflow1.14.0、Python 3.6、Gensim4.1.2
操作系统	Linux 3.10.0-693.el7.x86_64

4.2 数据集

4.2.1 基于序列的函数级源代码漏洞检测方案数据集

本方案采用的数据集来自 μ VulDeePecker 检测器所构建的开源漏洞数据集，原研究团队在 Software Assurance Reference Dataset (SARD) ^[58] 和 National Vulnerability Database (NVD) ^[59] 两个漏洞数据库进行了源代码的采集与标记，针对与库函数或 API 调用相关类型的漏洞，总共收集了包含 40 种类型漏洞的 33409 个 C/C++ 源代码。其中，漏洞代码按照其对应的漏洞类型分别标记为 1 至 40，非漏洞代码被标记为 0。表 4-2 展示了该数据集的相关细节。

表 4-2 数据集包含漏洞类型及标签对应情况表

Tab4-2 Dataset contains vulnerability types and label correspondence table

标签	漏洞类型	标签	漏洞类型
1	CWE-404	21	CWE-170
2	CWE-476	22	CWE-676
3	CWE-119	23	CWE-187
4	CWE-706	24	CWE-138
5	CWE-670	25	CWE-369
6	CWE-673	26	CWE-662,CWE-573
7	CWE-119,CWE-666,CWE-573	27	CWE-834
8	CWE-573	28	CWE-400,CWE-665
9	CWE-668	29	CWE-400,CWE-404
10	CWE-400,CWE-665,CWE-020	30	CWE-221
11	CWE-662	31	CWE-754
12	CWE-400	32	CWE-311
13	CWE-665	33	CWE-404,CWE-668
14	CWE-020	34	CWE-506
15	CWE-074	35	CWE-758
16	CWE-362	36	CWE-666
17	CWE-191	37	CWE-467
18	CWE-190	38	CWE-327
19	CWE-610	39	CWE-666,CWE-573
20	CWE-704	40	CWE-469

4.2.2 基于图与序列的函数级源代码漏洞检测方案数据集

本方案基于 7 个不同的 c/c++源代码数据集开展实验，其中 4 个为简单数据集，它们分别为 CWE-362，CWE-476，CWE-754 与 CWE-758。其从代码结构和功能相对简单的软件中收集，包含可能更容易识别的漏洞模式。剩余 3 个为复杂数据集，分别为 Big-Vul^[60]，Reveal^[61]与 FFMpeg+Qemu^[62]。Big-Vul 数据集涵盖了 2002 年至 2019 年的 CWE。在函数粒度上，数据集包含超过 10000 个漏洞函数。Reveal 数据集包含超过 18000 个函数，其中 9.16%是漏洞函数。FFMPeg+Qemu^[55]数据集包含超过 22000 个函数，其中 45.0%的为漏洞函数。表 4-3 展示了该数据集的相关细节。

表 4-3 7 个数据集

Tab4-3 7 datasets

数据集	漏洞数	非漏洞数	比率（漏洞数：非漏洞数）
CWE-362	189	375	1: 2
CWE-476	396	1227	1: 3.1
CWE-754	1359	3684	1: 2.7
CWE-758	367	995	1: 2.7
Big-Vul	10547	168752	1: 16
Reveal	1664	16505	1: 9.9
FFMPeg+Qemu	10067	12294	1: 1.2

4.3 评估指标

4.3.1 基于序列的函数级源代码漏洞检测方案评估指标

现设 M 为包含上述 40 种类型的漏洞样本集合， $|M| = 40$ ， m 作为下标标记第 m 类漏洞， N_m 代表类型为 m 的漏洞样本集合， $|N_m|$ 为类型为 m 的漏洞样本数量，以下所有以 m 为下标的标记都代表针对类型为 m 的漏洞。现在记， TP_m 表示被正确分类为漏洞类型 m 的 m 类漏洞样本数量； FP_m 表示被分类为漏洞类型 m 的无漏洞样本数量，即误报； FN_m 表示被分类为无漏洞类型的 m 类漏洞样本数量，即漏报； TN_m 表示被正确分类为无漏洞类型的无漏洞样本数量。本漏洞检测方案采用以下常用的深度学习模型评估指标，具体的计算公式如下：

①算术平均误报率 $\widehat{FPR}$ ，即对每个漏洞类型的检测误报率求算术平均数，误报率是指误报的样本数量与无漏洞的样本数量的比值，具体计算公式如下：

$$\widehat{FPR} = \frac{1}{|M|} \sum_{m \in M}^M \frac{FP_m}{FP_m + TN_m} \quad (4-1)$$

②算术平均漏报率 $\widehat{FNR}$ ，即对每个漏洞类型的漏报率求算术平均数，漏报率是指漏报的样本数量与有漏洞样本数量的比值，具体的计算公式如下：

$$\widehat{FNR} = \frac{1}{|M|} \sum_{m \in M}^M \frac{FN_m}{TP_m + FN_m} \quad (4-2)$$

③算术平均精确率 $\hat{P}$ ，即对每个漏洞类型的检测精确率求算术平均数，精确率是指正确预测的漏洞样本数量与所有被分类为漏洞样本数量的比值，具体的计算公式如下：

$$\hat{P} = \frac{1}{|M|} \sum_{m \in M} \frac{TP_m}{TP_m + FP_m} \quad (4-3)$$

④算术平均召回率 $\hat{R}$ ，即对每个漏洞类型的检测召回率求算术平均数，召回率是指正确预测的漏洞样本数量与所有漏洞样本数量的比值，具体的计算公式如下：

$$\hat{R} = \frac{1}{|M|} \sum_{m \in M} \frac{TP_m}{TP_m + FN_m} \quad (4-4)$$

⑤算术平均 F1 指数 $\hat{F}_1$ ，即对每个漏洞类型的 F1 指数的算术平均数，F1 指数是根据准确率与召回率所给出的一种综合评价，是二者的调和平均值。具体的计算公式如下：

$$\hat{F}_1 = \frac{1}{|M|} \sum_{m \in M} \frac{2 * P_m * R_m}{P_m + R_m} \quad (4-5)$$

其中，

$$P_m = \frac{TP_m}{TP_m + FP_m} \quad (4-6)$$

$$R_m = \frac{TP_m}{TP_m + FN_m} \quad (4-7)$$

⑥加权平均误报率 $\overline{FPR}$ ，即对每个漏洞类型的误报率求加权平均数，具体的计算公式如下：

$$\overline{FPR} = \frac{1}{\sum_{m \in M} |N_m|} \sum_{m \in M} |N_m| * \frac{FP_m}{FP_m + TN_m} \quad (4-8)$$

⑦加权平均漏报率 $\overline{FNR}$ ，即对每个漏洞类型的漏报率求加权平均数，具体的计算公式如下：

$$\overline{FNR} = \frac{1}{\sum_{m \in M} |N_m|} \sum_{m \in M} |N_m| * \frac{FN_m}{TP_m + FN_m} \quad (4-9)$$

⑧加权平均 F1 指数 $\tilde{F}_1$ ，即对每个漏洞类型的 F1 指数的加权平均数，具体的计算公式如下：

$$\tilde{F}_1 = \frac{1}{\sum_{m \in M} |N_m|} \sum_{m \in M} |N_m| * \frac{2 * P_m * R_m}{P_m + R_m} \quad (4-10)$$

在具体的实验环节中，我们将针对每一类型的漏洞分别进行相关指标的计算，最终求取算术平均值。对于加权平均指标的计算，还需要额外通过事先统计每个类型漏洞样本数量在总体漏洞样本数量中的占比，来获取对应类型漏洞的权重。综合考虑两种平均指标，可以更科学的度量漏洞检测模型的整体性能。

4.3.2 基于图与序列的函数级源代码漏洞检测方案评估指标

本文采用 4 个指标对模型进行评估, 即漏报率(FNR)、误报率(FPR)、准确率(Accuracy)、及 F1 指数(F1_score), 具体计算公式分别如下:

$$FNR = \frac{FN}{TP + FN} \quad (4-11)$$

$$FPR = \frac{FP}{FP + TN} \quad (4-12)$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4-13)$$

$$F1_{score} = \frac{2 * Precision * Recall}{Precision + Recall} \quad (4-14)$$

其中,

$$Precision = \frac{TP}{TP + FP} \quad (4-15)$$

$$Recall = \frac{TP}{TP + FN} \quad (4-16)$$

TP、TN、FP 以及 FN 是组成以上公式的基础部分, 其中, TP 表示将正样本预测为正样本的数量, TN 表示将正样本预测为负样本的数量, FP 表示将负样本预测为正样本的数量, 即误报, FN 表示将正样本预测为负样本的数量, 即漏报。因此, 漏报率(FNR)表示漏报的样本占正样本数量的比例, 误报率(FPR)表示误报的样本占负样本数量的比例, 准确率(Accuracy)表示预测正确的数据占总样本的比例, F1 指数(F1_score)是根据准确率(Precision)与召回率(Recall)给出的一种综合评价, 是二者的调和平均值。

4.4 实验与结果分析

4.4.1 基于序列的函数级源代码漏洞检测方案实验设计

本方案的实验将以回答以下研究问题的形式设计呈现:

研究问题 1: 该漏洞检测方案能否有效识别漏洞?

从多类型漏洞数据集中选取 CWE-119 漏洞数据样本进行实验, 验证方案针对单一类型漏洞的识别性能。选取误报率、漏报率与 F1 指数作为模型评估指标。详细设计如下:

实验组 1: HAN(GRU)+TextCNN

实验组 2: HAN(LSTM)+TextCNN

对照组: VulDeePecker^[33]

此外,为进行时间成本的比较,从多类型漏洞数据集中选取 6 种类型漏洞进行小规模实验,选取的具体类型为 CWE-119、CWE-758、CWE-74、CWE-706、CWE-400 与 CWE-573 对应的漏洞样本数目分别为 10440、7660、4021、3550、916 与 142。详细设计如下:

实验组 1: HAN(GRU)+TextCNN; 实验组 2: HAN(LSTM)+TextCNN; 对照组 1: μ VulDeePecker^[19]; 对照组 2: VulDeePecker+^[33]; 对照组 3: RNN^[44]; 对照组 4: LSTM^[44]; 对照组 5: Text-CNN^[44]; 对照组 6: VulDeePecker^[33]

其中,对照组 2 在初始模型 VulDeePecker 的基础上修改分类器为 softmax,对照组 3、4、5 使用原文献模型参数,对照组 6 采用 VulDeePecker 分别对六种类型的漏洞进行二分类训练,采用漏报率、误报率与 F1 指数作为二分类的评估指标。

研究问题 2: 输入层选用的 4 种词向量技术是否影响方案的漏洞检测性能,哪种方法可取得最佳性能?

基于以上包含 6 种类型漏洞的小规模数据集开展实验,选取算术平均评价指标对模型性能进行综合评估。详细设计如下:

实验组 1: FastText+HAN(GRU)+TextCNN

实验组 2: GloVe+HAN(GRU)+TextCNN

实验组 3: ELMo+HAN(GRU)+TextCNN

对照组 1: word2vec+HAN(GRU)+TextCNN

实验组 4: FastText+HAN(LSTM)+TextCNN

实验组 5: GloVe+HAN(LSTM)+TextCNN

实验组 6: ELMo+HAN(LSTM)+TextCNN

对照组 2: word2vec+HAN(LSTM)+TextCNN

研究问题 3: 该漏洞检测方案在多类型数据集上与现有软件漏洞检测工具相比性能如何?

基于完整数据集开展多类型漏洞检测实验，本次实验采取算术平均指标与加权平均指标相结合的方式对模型进行度量。详细设计如下：

实验组 1: HAN(GRU)+TextCNN

对照组 1: μ VulDeePecker^[19]

对照组 2: VulDeePecker+^[33]

对照组 3: AVDetect^[32]

对照组 4: Cai^[63]

研究问题 4: 与现有其他基于序列的漏洞检测方案配合实验能否有效提升模型性能?

基于完整数据集，选取该方案与现有性能最佳的 μ VulDeePecker 检测器进行配合实验，与两个模型单独实验结果进行对比。其中，配合算法如下：设本文模型将某一数据样本确定为类型为 a 的漏洞的概率为 p_1 ， μ VulDeePecker 模型将同一数据样本确定为类型为 b 的漏洞的概率为 p_2 ，若 $p_1 > p_2$ ，则最终认定漏洞类型为 a；反之，认定漏洞类型为 b。即取概率较大的模型判定作为两个模型配合检测的综合判定结果。详细设计如下：

实验组 1: 两方案配合。

对照组 1: HAN(GRU)+TextCNN

对照组 2: μ VulDeePecker^[19]

4.4.2 基于序列的函数级源代码漏洞检测方案结果与分析

(1) 研究问题 1 的实验结果：

表 4-4 与表 4-5 展示了问题 1 所对应的实验结果。实验结果显示，对比常用的单一结构的检测模型，即对照组 3、4、5，本模型的检测性能有着较高的提升，实验组 1 的 F1 指数分别提高了 38.58%、29.38%、20.81%，实验组 2 的 F1 指数分别提高了 39.86%、30.66%、22.09%，均有着较大的增加；针对独立训练的二分类模型，在 CWE-758、CWE-74、CWE-706 与 CWE-400 有着高于 85% 的 F1 指数，而对于样本数量最大的 CWE-119 与样本数量不足的 CWE-573 则检测性能较差，尤其是 CWE-573，F1 指数不足 60%，且漏报率接近 50%，其平均 F1 指数仅为 82.1%。对照组 1 的 μ VulDeePecker

与对照组 2 的 VulDeePecker+的 $\hat{F}_1$ 指数分别为 93.20%与 86.64%，检测性能明显优于单独的漏洞分类器。基于 bi-GRU 的 HAN+TextCNN 模型的 $\hat{F}_1$ 指数达 95.12%，检测性能与 μ VulDeePecker 模型十分接近。基于 bi-LSTM 的 HAN+TextCNN 模型的 $\hat{F}_1$ 指数达 95.12%，较对照组分别提高了 0.92%、7.48%与 12.02%，其是具有最优检测性能的模型。从检测时间角度进行分析，实验组 1、2 对应的模型检测时间略低于 μ VulDeePecker 检测器，略高于 VulDeePecker+检测器。综合评估，得出结论：HAN+TextCNN 模型可以进行多类型的漏洞检测，且具有较优性能，从检测效率与检测能力两方面综合考虑，模型检测性能得到进一步提升。

表 4-4 可行性实验结果 1

Tab4-4 Feasibility experiment results1

模型	$FPR(\%)$	$FNR(\%)$	$F1(\%)$	测试时间(s)
实验组 1	0.08	5.16	90.84	3009.63
实验组 2	0.04	3.99	92.12	3010.12
对照组 1	3.07	5.22	81.00	2809.10

表 4-5 可行性实验结果 2

Tab4-5 Feasibility experiment results2

模型	$\widehat{FPR}(\%)$	$\widehat{FNR}(\%)$	$\widehat{F}_1(\%)$	测试时间(s)
实验组 1	0.06	5.16	92.84	1517.56
实验组 2	0.04	3.79	94.12	1618.69
对照组 1	0.04	4.00	93.20	1713.13
对照组 2	0.10	15.51	86.64	1123.12
对照组 3	31.12	26.30	54.26	1769.33
对照组 4	17.23	20.17	63.46	1898.36
对照组 5	11.09	15.36	72.03	1428.59
对照组 6				
数据集	$FPR(\%)$	$FNR(\%)$	$F1(\%)$	测试时间(s)
CWE-119	3.07	5.22	81.00	2809.10
CWE-758	7.55	11.87	90.07	1466.10
CWE-74	1.00	4.30	87.59	1201.60
CWE-706	1.18	4.00	90.63	1065.30
CWE-400	0.04	22.56	86.23	1077.60
CWE-573	0.01	45.31	57.39	1199.50
综合	0.21	15.54	82.15	8819.20

(2) 研究问题 2 的实验结果:

表 4-6 展示了问题 2 所涉对应的实验结果。实验结果显示，不同的词向量模型会对漏洞检测性能产生影响。两种不同的网络架构在由 word2vec 模型转换为 FastText 模型

时, $\hat{F}_1$ 指数均略微下降, 其中实验组 1 下降 0.18%, 实验组 4 下降 0.03%; 两种不同的网络架构在由 word2vec 模型转换为 GloVe 模型时, $\hat{F}_1$ 指数均有所提升, 其中实验组 2 提升 1.28%, 实验组 5 提升 1.77%; 两种不同的网络架构在由 word2vec 模型转换为 ELMo 模型时, $\hat{F}_1$ 指数均略微下降, 其中实验组 3 下降 0.36%, 实验组 6 提升 0.11%。两种模型架构对应的 GloVe 词向量模型均取得最好的检测性能。综合评估, 得出结论: 通过合理改进词向量模型, 应用更先进的词向量技术, 使代码序列在转换成词向量时融入更多的上下文信息, 能偶改进漏洞检测模型的性能。

表 4-6 词向量实验结果

Tab4-6 Experimental results of word vector comparison

模型	$FPR(\%)$	$FNR(\%)$	$\hat{F}_1(\%)$
实验组 1	0.08	5.55	92.66
实验组 2	0.04	3.88	94.12
实验组 3	0.04	4.00	93.20
对照组 1	0.06	5.16	92.84
实验组 4	0.06	4.02	94.09
实验组 5	0.03	3.02	95.89
实验组 6	0.03	3.77	94.23
对照组 2	0.04	3.79	94.12

(3) 研究问题 3 的实验结果:

表 4-7 展示了问题 3 所对应的实验结果。实验结果显示, 对照组 1, 即 μ VulDeePecker 模型在各项评估指标上具有最优性能。本研究模型在所有评估指标上都有着接近 μ VulDeePecker 模型的检测性能。本研究方案模型算术平均 F1 指数 $\hat{F}_1$ 较对照组 2、对照组 3 与对照组 4 分别提升了 7.1%、45.36%与 7.6%, 加权平均 F1 指数 $\tilde{F}_1$ 较对照组 2、对照组 3 与对照组 4 分别提升了 1.29%、46.2%与 6.22%。较现有的绝大多数方案, 均有了较大性能上的提升。综合评估, 得出结论: 本研究方案模型对多类型的漏洞检测较为有效, 模型中嵌入的双层注意力机制可以有效辅助模型进行类型识别, 提升检测性能。

表 4-7 有效性实验结果

Tab4-7 Effectiveness experiment results

模型	$\hat{F}_1(\%)$	$\widetilde{F}_1$
实验组	89.66	91.41
对照组 1	91.84	93.01
对照组 2	83.56	90.12
对照组 3	44.30	45.21
对照组 4	83.06	85.19

(4) 研究问题 4 的实验结果:

表 4-8 展示了问题 4 所对应的实验结果。表中的实验结果显示, 相比于对照组 1 和对照组 2, $\hat{F}_1$ 的值分别提高了 2.7%和 0.52%, $\widetilde{F}_1$ 的值分别提高了 3.73%和 2.13%。由此证明, 模型的结合使用可以提升整体的漏洞检测性能, 可以认为, 优秀模型的配合使用可以弥补多者之间对不同类型漏洞存在的检测缺陷, 从而精进模型性能。

表 4-8 配合性实验结果

Tab4-8 Compatibility test results

模型	$\hat{F}_1(\%)$	$\widetilde{F}_1$
实验组	92.36	95.14
对照组 1	89.66	91.41
对照组 2	91.84	93.01

4.4.3 基于图与序列的函数级源代码漏洞检测方案实验设计

本方案的实验将以回答以下研究问题的形式设计呈现:

研究问题 1: 该漏洞检测方案与现有软件漏洞检测工具相比性能如何?

对照组 1: AVDetect^[32]、VulDeePecker^[33]、Codebert^[57]

对照组 2: Devign^[55]、Reveal^[34]、IVDetect^[64]、FUNDED^[46]

对照组 3: DeepVulSeeker^[65]

以上实验方案均采用基于深度学习的漏洞检测技术, 其中对照组 1 中的模型采用基于序列的方法, 对照组 2 中的模型采用基于图的方法, 而对照组 3 中的模型采用基于图与序列相结合的方法。

研究问题 2: 综合考虑图与序列信息是否有助于提升模型漏洞检测性能?

本研究的漏洞检测方案抽取的信息类型包括图节点：节点类型与节点元素、图边：边类型与边方向及序列信息，为验证综合考虑多类型信息对漏洞检测模型的有效性，采取控制变量的方法，减少对照组中抽取的信息类型。

对照组 1：仅抽取图信息；对照组 2：仅抽取序列信息。

研究问题 3：不同的节点初始化嵌入方法是否影响方案的漏洞检测性能？

本文漏洞检测方案采用预训练模型 CodeBert 对节点元素进行初始化并通过再次训练微调嵌入权重。通过设计对照实验与其他图节点向量化方法进行比较，具体对照组设置如下：

对照组 1：Word2vec；对照组 2：Dov2vec。

研究问题 4：不同的序列初始化嵌入方法是否影响方案的漏洞检测性能？

本文漏洞检测方案采用预训练模型 CodeBert 对序列标记进行初始化并通过再次训练微调嵌入权重。通过设计对照实验与传统的词向量模型进行比较，具体对照组设置如下：

对照组 1：Word2vec；对照组 2：GloVe；对照组 3：Fasttext。

4.4.4 基于图与序列的函数级源代码漏洞检测方案结果与分析

（1）研究问题 1 的实验结果：

表 4-9 与表 4-10 展示了问题 1 所涉及的有关对照组 1 的实验结果。在简单数据集上，本研究模型在四个数据集上的平均准确率与 F1 值均高于对照组所有模型。而在真实世界数据集上，实验组除在 FFMpeg+Qemu 上的 F1 值略低于 VulDeePecker^[33]与 codebert^[57]，其他各项结果均高于对照组结果。

表 4-9 对照组 1 实验结果 1

Tab4-9 Control Group 1 Experimental Result 1

数据集 方案	CWE-362		CWE-476		CWE-754		CWE-758	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1
AVDetect ^[32]	0.6167	0.5136	0.5819	0.3812	0.7371	0.3430	0.6731	0.2031
VulDeePecker ^[33]	0.9500	0.9048	0.8070	0.8932	0.9574	0.9351	0.7887	0.8818
codebert ^[57]	0.7691	0.8695	0.9083	0.8387	0.5246	0.4706	0.8356	0.7177
实验组	0.9201	0.8783	0.9169	0.8515	0.9899	0.9811	0.9717	0.9636

表 4-10 对照组 1 实验结果 2

Tab4-10 Control Group 1 Experimental Result 2

数据集 方案	Big-Vul		Reveal		FFMPeg+Qemu	
	ACC	F1	ACC	F1	ACC	F1
AVDetect ^[32]	0.2105	0.2563	0.1502	0.1652	0.5693	0.3336
VulDeePecker ^[33]	0.1582	0.1970	0.1569	0.1752	0.5809	0.5922
codebert ^[57]	0.3003	0.3112	0.2017	0.2255	0.5309	0.6632
实验组	0.5934	0.6303	0.6533	0.6852	0.6399	0.5293

表 4-11 与表 4-12 展示了问题 1 所涉及的有关对照组 2 的实验结果。在简单数据集上，本研究模型在四个数据集上的平均准确率与 F1 值均高于对照组所有模型。而在真实世界数据集上，实验组除在 FFMPeg+Qemu 数据集上的 F1 值略低于 IVDetect^[64]与 FUNDED^[46]，其他各项结果均高于对照组结果。

表 4-11 对照组 2 实验结果 1

Tab4-11 Control Group 2 Experimental Result 1

数据集 方案	CWE-362		CWE-476		CWE-754		CWE-758	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1
Devign ^[55]	0.8571	0.8148	0.8250	0.5333	0.9291	0.8794	0.8822	0.8164
Reveal ^[34]	0.8637	0.8201	0.8356	0.7233	0.9300	0.8952	0.9001	0.8362
IVDetect ^[64]	0.9013	0.8666	0.8510	0.8423	0.9366	0.8855	0.9106	0.8763
FUNDED ^[46]	0.9446	0.9521	0.8889	0.9087	0.9286	0.9331	0.9583	0.9615
实验组	0.9201	0.8783	0.9169	0.8515	0.9899	0.9811	0.9717	0.9636

表 4-12 对照组 2 实验结果 2

Tab4-12 Control Group 2 Experimental Result 2

数据集 方案	Big-Vul		Reveal		FFMPeg+Qemu	
	ACC	F1	ACC	F1	ACC	F1
Devign ^[55]	0.2230	0.2600	0.3011	0.3211	0.5972	0.4630
Reveal ^[34]	0.2864	0.3011	0.3521	0.4026	0.5264	0.4912
IVDetect ^[64]	0.4013	0.3510	0.5244	0.4531	0.6731	0.6520
FUNDED ^[46]	0.4766	0.4000	0.5299	0.4917	0.5695	0.7140
实验组	0.5934	0.6303	0.6533	0.6852	0.6399	0.5293

表 4-13 与表 4-14 展示了问题 1 所涉及的有关对照组 3 的实验结果。在简单数据集上，本研究模型在 CWE-362 与 CWE-476 上的两项指标均高于对照组模型，在 CWE-754 与 CWE-758 上的两项指标虽然相较略低，但两项指标均值仍分别达到 0.9808 与 0.9724。而在真实世界数据集上，实验组除在 FFMPeg+Qemu 上的 F1 值略低以外，其他各项结果均高于对照组结果。

表 4-13 对照组 3 实验结果 1

Tab4-13 Control Group 3 Experimental Result 1

数据集 方案	CWE-362		CWE-476		CWE-754		CWE-758	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1
DeepVulSeeker ^[65]	0.9123	0.8387	0.9080	0.8052	0.9999	0.9999	0.9927	0.9859
实验组	0.9201	0.8783	0.9169	0.8515	0.9899	0.9811	0.9717	0.9636

表 4-14 对照组 3 实验结果 2

Tab4-14 Control Group 3 Experimental Result 2

数据集 方案	Big-Vul		Reveal		FFMPeg+Qemu	
	ACC	F1	ACC	F1	ACC	F1
DeepVulSeeker ^[65]	0.5689	0.5986	0.6023	0.6431	0.6382	0.5998
实验组	0.5934	0.6303	0.6533	0.6852	0.6399	0.5293

综上所述，得出结论 1。

结论 1：与现有方案相比，在两类数据集上，本研究的漏洞检测方案都有更卓越的性能。

(2) 研究问题 2 的实验结果：

表 4-15 与表 4-16 展示了问题 2 对应的实验结果。对照组 1（即单独抽取图表示）的整体性能较对照组 2（即单独抽取序列表示）更好，但实验组的结果在两类数据集上，准确率与 F1 值均有更明显提高。以上差距验证了图结构信息与序列语义信息对漏洞检测的重要性。

表 4-15 研究问题 2 实验结果 1

Tab4-15 Research Question 2 Experimental Result 1

数据集 方案	CWE-362		CWE-476		CWE-754		CWE-758	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1
对照组 1	0.7964	0.7525	0.7748	0.7127	0.8517	0.8500	0.8334	0.8267
对照组 2	0.6102	0.5643	0.6001	0.5597	0.6322	0.6221	0.6286	0.6201
实验组	0.9201	0.8783	0.9169	0.8515	0.9899	0.9811	0.9717	0.9636

表 4-16 研究问题 2 实验结果 2

Tab4-16 Research Question 2 Experimental Result 2

数据集 方案	Big-Vul		Reveal		FFMPeg+Qemu	
	ACC	F1	ACC	F1	ACC	F1
对照组 1	0.5566	0.6062	0.6086	0.6200	0.5937	0.5019
对照组 2	0.4577	0.5000	0.5097	0.5211	0.4963	0.4128
实验组	0.5934	0.6303	0.6533	0.6852	0.6399	0.5293

基于以上分析，得出结论 2。

结论 2：较单独考虑图或单独考虑序列，综合考虑二者的漏洞检测方案有着更好的检测性能。

（3）研究问题 3 的实验结果：

表 4-17 与表 4-18 展示了问题 3 对应的实验结果。在两类数据集上，实验组都具有更高的准确率与 F1 值，其中，较对照组 1，在简单数据集上，准确率差异的平均值达到 0.1723，F1 值差异的平均值达到 0.1744；在复杂数据集上，准确率差异的平均值达到 0.1394，F1 值差异的平均值达到 0.1165。较对照组 2，在简单数据集上，准确率差异的平均值达到 0.1301，F1 值差异的平均值达到 0.1337；在复杂数据集上，准确率差异

的平均值达到 0.1101，F1 值的差异的平均值达到 0.0915。以上差距验证了本文所使用的节点嵌入方式能有效替代传统的词向量方法，具有一定的合理性。

表 4-17 研究问题 3 实验结果 1

Tab4-17 Research Question 3 Experimental Result 1

数据集 方案	CWE-362		CWE-476		CWE-754		CWE-758	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1
对照组 1	0.7712	0.7123	0.7585	0.6985	0.7977	0.7923	0.7820	0.7743
对照组 2	0.8019	0.7465	0.7985	0.7326	0.8412	0.8367	0.8365	0.8241
实验组	0.9201	0.8783	0.9169	0.8515	0.9899	0.9811	0.9717	0.9636

表 4-18 研究问题 3 实验结果 2

Tab4-18 Research Question 3 Experimental Result 2

数据集 方案	Big-Vul		Reveal		FFMPeg+Qemu	
	ACC	F1	ACC	F1	ACC	F1
对照组 1	0.4562	0.4863	0.5121	0.5321	0.5001	0.4769
对照组 2	0.4952	0.5017	0.5347	0.5698	0.5263	0.4989
实验组	0.5934	0.6303	0.6533	0.6852	0.6399	0.5293

基于以上分析，得出结论 3。

结论 3：与传统的节点源代码初始化嵌入方式相比，本研究所采用的方法能更好的提取局部语义信息，提升方案漏洞检测性能。

（4）研究问题 4 的实验结果：

表 4-19 与表 4-20 展示了问题 4 对应的实验结果。所有对照组中，对照组 2（即使用 GloVe 模型）的整体性能最好，然而，实验组的结果显示，在简单数据集上，准确率平均提高了 0.0611，F1 值平均提高了 0.0707；在真实世界数据集上，准确率平均提高了 0.0658，F1 值平均提高了 0.0663。以上差距验证了预训练模型的引入较传统词向量模型对序列的嵌入有更好的效果，具有一定的合理性。

表 4-19 研究问题 4 实验结果 1

Tab4-19 Research Question 4 Experimental Result 1

数据集 方案	CWE-362		CWE-476		CWE-754		CWE-758	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1
对照组 1	0.7852	0.7554	0.7689	0.7356	0.8077	0.7989	0.7961	0.7533
对照组 2	0.8865	0.8213	0.8512	0.7961	0.9176	0.9033	0.8991	0.8709
对照组 3	0.7963	0.7655	0.7861	0.7399	0.8272	0.8043	0.8054	0.7856
实验组	0.9201	0.8783	0.9169	0.8515	0.9899	0.9811	0.9717	0.9636

表 4-20 研究问题 4 实验结果 2

Tab4-20 Research Question 4 Experimental Result 2

数据集 方案	Big-Vul		Reveal		FFMPeg+Qemu	
	ACC	F1	ACC	F1	ACC	F1
对照组 1	0.5063	0.5217	0.5625	0.5899	0.5381	0.4011
对照组 2	0.5242	0.5534	0.5896	0.6002	0.5755	0.4923
对照组 3	0.5081	0.5246	0.5711	0.5934	0.5303	0.4087
实验组	0.5934	0.6303	0.6533	0.6852	0.6399	0.5293

基于以上分析，得出结论 4。

结论 4：与传统的源代码序列初始化嵌入方式相比，本研究所采用的针对代码的预训练模型能更好的提取抽取代码全局语义信息，进而提升模型的整体漏洞检测性能。

4.5 本章小结

本章首先对两种漏洞检测方案的实验环境进行了介绍。之后，分别两种方案开展实验所使用的数据集与评估指标进行了详细的说明。最后，以提出研究问题的形式针对两类漏洞检测方案设计并开展了验证实验，分析实验结果并得出结论。

第五章 总结与展望

5.1 全文工作总结

漏洞检测任务是人们从事网络安全领域难以避免的挑战，软件每时每刻都在进行更新，新式的漏洞层出不穷且无穷无尽，这就对漏洞检测任务的效率提出了更高的要求，如何在降低从业者的劳动参与提高漏洞检测精度寻求双赢是目前从业者所关注的重点研究领域。深度学习技术能够借助合理设计隐藏层的深度挖掘深层信息，具有强大的信息提取能力，近年来在漏洞检测领域已经有所作为。

本文针对以上方向，提出了两种针对不同代码表示形式的漏洞检测方案。

一、基于序列的函数级源代码漏洞检测方案：构建了一个融合多种词向量技术并引入分层注意力机制的 HAN-TextCNN 神经网络架构，实现了多类型漏洞的识别与检测。实验结果表明，本方案的检测性能在多个评估指标上有所提升的同时，降低了漏洞检测的时间成本。其中，多层注意力机制的引入有效提升了漏洞类型的识别能力。

二、基于图与序列的函数级源代码漏洞检测方案：1)综合学习程序语义和结构信息来识别漏洞，以自动化的方式捕获了比现有方法更综合的特征。2)在多环节引入基于代码的预训练语言模型来代替传统的词向量模型进行信息编码。使用 CodeBert 生成源代码序列的词向量矩阵,使用 CodeBert 进行节点元素初始化并用 BiLSTM 聚合节点内部的局部语义信息。3) 基于漏洞数据集与现有检测方案进行了对照实验。实验结果表明，该方案在多个评估指标上显著优于现有的漏洞检测方法。此外，对照试验也验证了各环节的有效性，并表明本研究对节点、边及序列信息的处理策略优于现有其他方案。

5.2 未来工作展望

本研究尚存在以下三点局限性：

(1) 无法精确的定位漏洞位置，受限于粒度，仅能在函数粒度上甄别漏洞，无法定位漏洞详细语句。

(2) 两种漏洞检测方案均侧重源代码层面上的漏洞检测，数据集均为 C/C++语言书写，语言类型单一，无法跨语言检测漏洞。

（3）基于图与序列的函数级源代码漏洞检测方案引入的预训练模型 CodeBert 含 12 个编码器层，参数多达 1.1 亿，训练与部署成本高昂。

以上局限也为未来的漏洞检测领域指明了潜在的研究方向，未来研究工作旨在完善以上局限：

（1）缩小检测粒度，进行语句级别的漏洞数据集构建，辅助训练，进而实现漏洞位置的精准定位。

（2）扩展数据集，加入更多种类的程序设计语言；从二进制代码层面进行漏洞挖掘。用以解决跨语言漏洞检测问题。

（3）提出一个针对代码语言的更轻量级预训练模型优化方案，降低漏洞检测模型的训练与部署成本。

参考文献

- [1] Wang M, Zhu T, Zhang T, et al. Security and privacy in 6g networks: new areas and new challenges[J]. Digital Communications and Networks, 2020, 6(3):281-291.
- [2] Miao Y, Chen C, Pan L, et al. Machine learning-based cyber attacks targeting on controlled information[J]. ACM Computing Surveys, 2022, 54(7):1-36.
- [3] Chen X, Li C, Wang D, et al. Android hiv: a study of repackaging malware for evading machine-learning detection[J]. IEEE Transactions on Information Forensics and Security, 2019, 15: 987–1001.
- [4] Zhang J, Chen X, Xiang Y, et al. Robust network traffic classification[J]. IEEE/ACM Transactions on Networking, 2014, 23(4):1257–1270.
- [5] Tassey, Gregory. The Economic Impacts of Inadequate Infrastructure for Software Testing[J]. National Institute of Standards & Technology, 2002, 15(3):125.
- [6] Venolia G. Maintaining mental models: a study of developer work habits[J]. Proceedings of the Icse '06 Acm, 2016:492-501.
- [7] Xu Z , Kremenek T , Zhang J .A Memory Model for Static Analysis of C Programs[C]//International Symposium On Leveraging Applications of Formal Methods. Verification and Validation. Springer. Berlin: Heidelberg, 2010.
- [8] Li Y, Chen B, Chandramohan M, et al. Steelix: Program-state based binary fuzzing[C]//Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. New York: Association for Computing Machinery, 2017:627–637.
- [9] Coverity: Coverity scan static analysis. 2022. <https://scan.coverity.com/>
- [10] KlocWork: Static code analysis for C, C++, C#, and Java. 2022. <https://www.perforce.com/products/klocwork>
- [11] LibFuzzer: A library for coverage-guided fuzz testing. 2022. <http://llvm.org/docs/LibFuzzer.html>
- [12] Cadar C, Dunbar D, Engler D. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs[C]//Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation. San Diego, CA, USA. 2008. 209–224.
- [13] Chipounov V, Kuznetsov V, Candea G. S2E: a platform for in-vivo multi-path analysis of software systems[J]. ACM SIGPLAN Notices, 2011, 47(4): 265-278.
- [14] 宋丛溪, 王辛, 张文喆. Angr 动态软件测试应用分析与优化[J]. 计算机工程与科学, 2018, 40(S1): 167-172. Song C X, Wang X, Zhang W Z. Analysis and optimization of ANGR in dynamic software test application[J]. Computer Engineering & Science, 2018, 40(S1):167-172.
- [15] Pan J, Yan G, Fan X. Digtool: A Virtualization-Based Framework for Detecting Kernel Vulnerabilities[C]// Proceedings of the 26th USENIX Security Symposium. Vancouver: USENIX, 2017. 149–165.
- [16] Rapidscan. 2022. <https://github.com/skavngr/rapidscan>

- [17] Duan X, Wu J Z, Ji S L, et al. VulSniper: Focus your attention to shoot fine-grained vulnerabilities[C]//Proceedings of the 28th International Joint Conference on Artificial Intelligence. Macao, China, 2019: 4665-4671.
- [18] Ghaffarian M S H R. Neural software vulnerability analysis using rich intermediate graph representations of programs[J]. Information Sciences: An International Journal, 2021, 553(1):189-207.
- [19] Zou D, Wang S, Xu S, et al. μ VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection[J]. IEEE Transactions on Dependable and Secure Computing, 2019, PP(99):1-1.
- [20] Li J, He P, Zhu J, et al. Software defect prediction via convolutional neural network[C]//Proceedings of the 2017 IEEE Int'l Conference. Software Quality, Reliability and Security (QRS). IEEE, 2017: 318-328.
- [21] Russell R, Kim L, Hamilton L, et al. Automated vulnerability detection in source code using deep representation learning[C]//Proceedings of the 17th IEEE international conference on machine learning and applications (ICMLA). Orlando, 2018:757-762.
- [22] Perl H, Dechand S, Smith M, et al. VCCFinder: Finding Potential Vulnerabilities in Open-Source Projects to Assist Code Audits[C]// Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS'15). ACM, 426-437.
- [23] Walden J, Stuckman J, Scandariato R. Predicting Vulnerable Components: Software Metrics vs Text Mining[C]//IEEE International Symposium on Software Reliability Engineering. IEEE, 2014:23-33.
- [24] Yamaguchi F, Wressnegger C, Gascon H. Chucky: Exposing missing checks in source code for vulnerability discovery[C] //Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security. 2013:499-510.
- [25] Pang Y, Xue X, Namin A S. Predicting vulnerable software components through N-Gram analysis and statistical feature selection[C] // Proceedings of IEEE International Conference on Machine Learning and Applications. 2015:543-548.
- [26] Zeng P, Lin G, Pan L, et al. Software vulnerability analysis and discovery using deep learning techniques: a survey[J]. IEEE Access, 2020, 8:197158-197172.
- [27] Singh S K, Chaturvedi A. Applying deep learning for discovery and analysis of software vulnerabilities: a brief survey[J]. Advances in Intelligent Systems and Computing, 2020, 1154: 649-658.
- [28] Sun N, Zhang J, Rimba P, et al. Data-driven cybersecurity incident prediction: a survey[J]. IEEE Communications Surveys & Tutorials, 2019, 21(2):1744-1772.
- [29] Qiu J, Zhang J, Luo W, et al. A survey of android malware detection with deep neural models[J]. ACM Computing Surveys, 2020, 53(6):1-36.
- [30] Lin G, Wen S, Han Q L, et al. Software vulnerability detection using deep neural networks: a survey[J]. Proceedings of the IEEE, 2020, 108(10):1825-1848.
- [31] Wang S, Liu T, Tan L. Automatically learning semantic features for defect prediction [C]//Proceedings of the 2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE).2016:297-308.

- [32] Russell R, Kim L, Hamilton L, et al. Automated vulnerability detection in source code using deep representation learning[C]//Proceedings of the 17th IEEE international conference on machine learning and applications (ICMLA). Orlando, 2018:757–762.
- [33] Li Z, Zou D, Xu S, et al. Vuldeepecker: a deep learning based system for vulnerability detection[C]//Network and Distributed Systems Security (NDSS) Symposium. San Diego, 2018:1-15.
- [34] Chakraborty, S, Krishna, R, Ding, Y, et al. Deep learning based vulnerability detection: are we there yet?[J]. IEEE Trans. Software. Engineering, 2020, 48 (9): 3280–3296.
- [35] Li X, Xin Y, Zhu H L, et al. Cross-domain vulnerability detection using graph embedding and domain adaptation[J]. Computers & Security, 2023,125: 103017.
- [36] Dong Y K, Tang Y, Cheng X T, et al. SedSVD: Statement-level software vulnerability detection based on Relational Graph Convolutional Network with subgraph embedding[J]. Information and Software Technology, 2023,158: 1-14.
- [37] Harer J A, Kim L Y, Russell L R, et al. Automated software vulnerability detection with machine learning[J/OL]. arXiv, 2018: 04497[2023-07-20]. <https://arxiv.org/abs/1803.04497>.
- [38] Lee Y J, Choi S H, Kim C, et al. Learning binary code with deep learning to detect software weakness[C]//Proceedings of the KSII the 9th international conference on internet (ICONI) Vientien, Laos, 2017.
- [39] Wu F, Wang J, Liu J, et al. Vulnerability detection with deep learning[C]//Proceedings of the 2017 3rd IEEE international conference on computer and communications (ICC C). Chengdu, 2017: 1298–1302.
- [40] Lin G, Zhang J, Luo W, et al. Software vulnerability discovery via learning multi-domain knowledge bases[J]. IEEE Transactions on Dependable and Secure Computing, 2021, 18(5): 2469–2485.
- [41] Shar L K, Tan H B K. Predicting common web application vulnerabilities from input validation and sanitization code patterns[C]//Proceedings of the 2012 27th IEEE/ACM international conference on automated software engineering. Germany, 2012: 310–313.
- [42] Lin G, Zhang J, Luo W, et al. Poster: vulnerability discovery with function representation learning from unlabeled projects[C]//Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. Dallas, 2017: 2539–2541.
- [43] Lin G, Zhang J, Luo W, et al. Cross-project transfer representation learning for vulnerable function discovery[J]. IEEE Transactions on Industrial Informatics, 2018, 14(7): 3289-3297.
- [44] Lin G, Xiao W, Zhang J, et al. Deep learning-based vulnerable function detection: a benchmark[C]//Proceedings of the International Conference on Information and Communications Security. Beijing:Springer, 2019: 219–232.
- [45] Li X, Xin Y, Zhu H L, et al. Cross-domain vulnerability detection using graph embedding and domain adaptation[J]. Computers & Security, 2023,125: 103017.
- [46] Wang, H, Ye, G, Tang, Z et al. Combining Graph-based Learning with Automated Data Collection for Code Vulnerability Detection[J]. IEEE Transactions on Information Forensics and Security, 2020, 16:1943-1958.

- [47] Dong Y K, Tang Y, Cheng X T, et al. SedSVD: Statement-level software vulnerability detection based on Relational Graph Convolutional Network with subgraph embedding[J]. Information and Software Technology, 2023,158: 1-14.
- [48] Yang Z, Yang D, Dyer C, et al. Hierarchical Attention Networks for Document Classification[C]//Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies.2016: 1480-1489.
- [49] Kim, Y. Convolutional Neural Networks for Sentence Classification[C]//Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP 2014), 1746 - 1751.
- [50] Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J. Distributed representations of words and phrases and their compositionality[J]. Advances in Neural Information Processing Systems, 2013, 26:3111-3119.
- [51] Pennington J , Socher R , Manning C .Glove: Global Vectors for WordRepresentation [C]//Conference on Empirical Methods in Natural Language Processing.2014:1532–1543.
- [52] JOULIN A, GRAVE E, BOJANOWSKI P, et al. Bag of tricks for efficient text classification[C]//Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics. Valencia, Spain, 2017: 427-431.
- [53] Peters M, Neumann M, Iyyer M, et al. deep Contextualized Word Representations[C] // Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics:Human Language Technologies. 2018(1):2227–2237.
- [54] Yamaguchi F, Golde N, Arp D, et al. Modeling and discovering vulnerabilities with code property graphs[C]//2014 IEEE Symposium on Security and Privacy. 2014: 590–604.
- [55] Zhou Y Q, Liu S Q, Siow J K, et al. Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks[J]. arXiv, 2019: 03496[2023-07-21]. <https://arxiv.org/abs/1909.03496>.
- [56] Sennrich R, Haddow B, Birch A. Neural machine translation of rare words with subword units[C]//Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics. Berlin: The Association for Computer Linguistics, 2016: 1-11.
- [57] Feng Z, Guo D, Tang D, et al. CodeBERT: a pre-trained model for programming and natural languages[J/OL]. arXiv, 2020:1-12[2023-7-20]. <https://arxiv.org/abs/2002.08155v4>.
- [58] SARD: Software assurance reference dataset. 2022. <https://samate.nist.gov/SRD/index.php>
- [59] NVD. 2022. <https://nvd.nist.gov/>
- [60] FAN J H, LI Y, WANG S H, et al. A C/C++ code vulnerability dataset with code changes and CVE summaries[C]//ACM 17th International Conference on Mining Software Repositories. IEEE,2020: 508-512.

- [61] Chakraborty S, Krishna R, Ding Y, et al. Deep learning based vulnerability detection: are we there yet?[J]. IEEE Transactions on Software Engineering, 2020, 48 (9): 3280–3296.
- [62] Zhou Y Q, Liu S Q, Siow J K, et al. Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks[J]. arXiv, 2019: 03496[2023-07-21]. <https://arxiv.org/abs/1909.03496>.
- [63] Cai, W J, Chen J L, Yu J P, et al. A Software Vulnerability Detection Method Based on Deep Learning with Complex Network Analysis and Subgraph Partition[J/OL]. Elsevier,2023:1-14[2023-7.21]. <https://ssrn.com/abstract=4422123>.
- [64] Li Y, Wang S, Nguyen T N. Vulnerability Detection with Fine-grained Interpretations [C]//Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE-2021). Athens, Greece, 2021: 1-12.
- [65] Wang J, Xiao H, Zhong S W, et al. DeepVulSeeker: A novel vulnerability identification framework via code graph structure and pre-training mechanism[J]. Future Generation Computer Systems,2023, 148: 15-26.

攻读硕士学位期间所取得的成就

- [1] 王守梁. 基于代码序列与图结构的源代码漏洞检测方案[J]. 中北大学学报（自然科学版），2023(6): 0-13.

致谢

2020 年 10 月,我带着本科毕业时的迷茫来到了中北大学大数据与网络安全实验室报道,正式成为了其中的一员。初来乍到,实验室融洽的氛围使性格稍具内向的我很自然的融入其中。研一期间,这里不仅有每周五晚上固定举办的学长学姐学习经验分享会,帮助我等学弟学妹们快速熟悉科研流程、了解研究方向、快速学术入门,还给我们每个新人提供了理论试讲的机会,参与讲解的过程,既提升了对知识的理解又增加了师生间的交流。研二期间,跟随导师去到威海,在导师与博士师兄的指导下投入到漏洞检测领域的研究工作当中。回顾整个研究生的学习与生活,心中倍感充实。

如今,如今,三年的研究生时光终于迎来了片尾曲,在不舍的同时,更多的是心怀感恩。首先,我要谢谢我的老师宋礼鹏教授,一直是他治学严谨、精益求精、锲而不舍的治学态度鼓舞了我,引导我步入了系统科学的研究领域,在我懈怠的时刻引导了我努力下去,当我在学习中遇到了困难的时候也倾囊相助。研二时期,跟随在导师身边的教学和生活经验,使我获益很大,使我由在科研上磕磕绊绊的系统科学小白,逐渐变成了掌握了一定程度的系统科学理论,能独立开展实验的学术入门者。论文写作后期,宋老师对我的研究提出了宝贵的修改意见,帮助我整理思路,保证了我论文的顺利完成,我对您的感激之情,溢于言表。此外,我还要感谢我同组的博士师兄朱宇辉与李靖,感谢师兄在研究生三年期间,牺牲自己做课题的宝贵时间给我提供学习生活上的帮助,不仅给我及时分享漏洞检测领域的论文研究,还在实验思路给我提供了莫大的帮助,感谢师兄在研二期间对我生活上的照顾,教会了很多生活技能,解决了我在生活上的诸多困扰,让我的科研工作没有了后顾之忧,祝二位未来的科研生活一切顺利,前程似锦。最后,我要着重感谢我的父母还有姥姥,虽然他们不能给我的科研工作带来帮助,但得益于他们在物质与精神上对我的全力支持,也得益于他们在我焦虑压力过大的时候对我的排忧解难,让我的研究生生活能够真正做到心无旁骛,专心致志。我会牢记研究生生活的点点滴滴,与你们所有人的相处给我二十多岁的黄金年龄阶段留下了宝贵的记忆。

感谢论文审核的专家与工作人员,万分感谢您们在繁忙的工作当中抽出身来阅读并且审核我的文章。最后,再次感谢所有中北大学大数据与网络安全实验室的师生,感谢研究生三年对我本人提供的帮助与支持。